THE SUCCESS NOTES

LINEAR DATA STRUCTURES

NOTES BY

Mr. H.R CHOUDHARY

Downloaded freely from
the-succes-notes.infy.uk

Topics to be covered
Stack

What do you mean by Stack ?

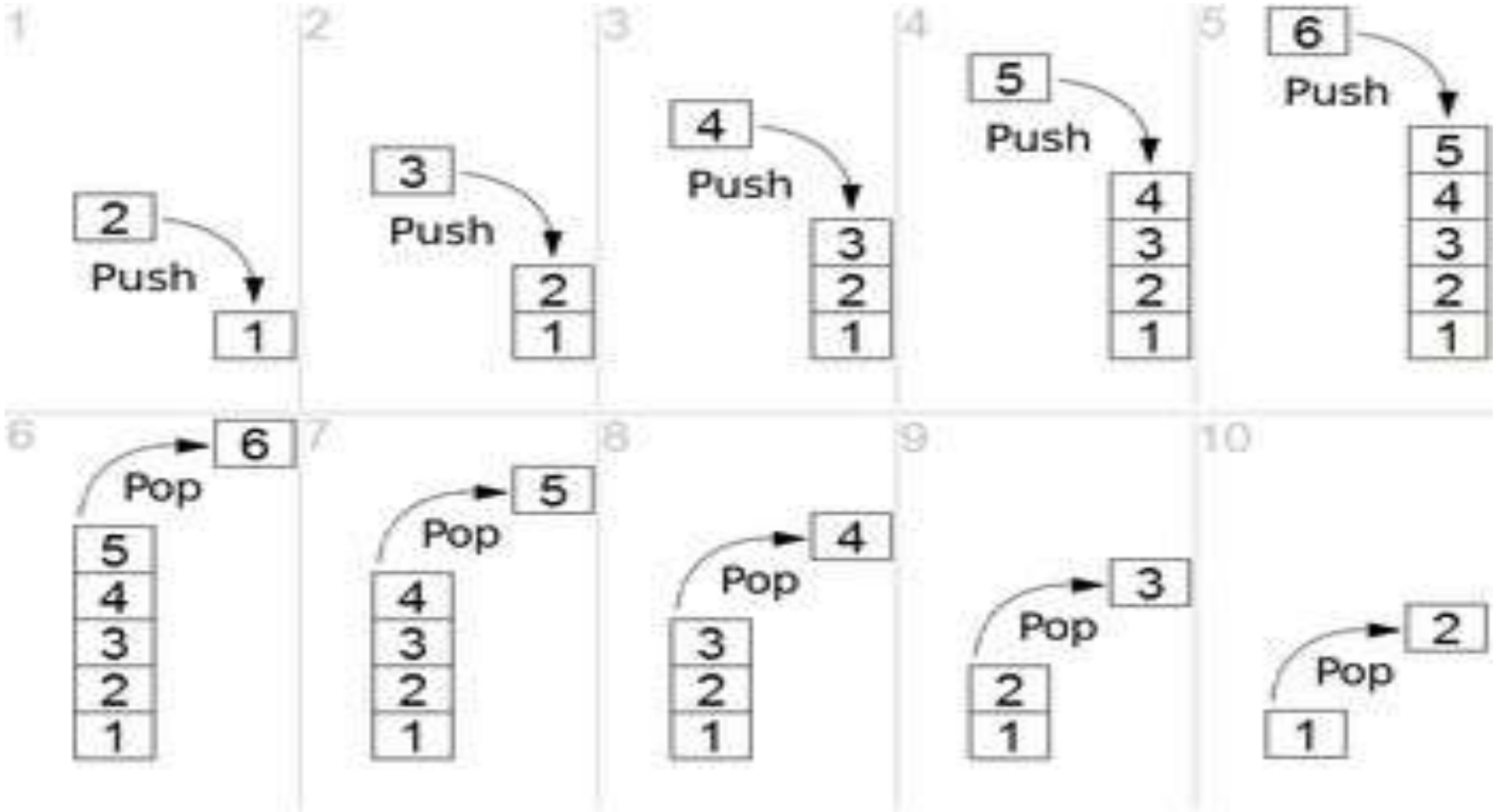
Stack Opertaion.

Different applications of Stack.

What do you mean by Stack ?

A stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc.

Example..


1
2
Push
1
2
3
Push
2
1
3
4
Push
3
2
1
4
5
Push
4
3
2
1
5
6
Push
5
4
3
2
1
6
6
Pop
5
4
3
2
1
7
5
Pop
4
3
2
1
8
4
Pop
3
2
1
9
3
Pop
2
1
10
2
Pop
1

Example..

LIFO (Last In First Out)

Stack..

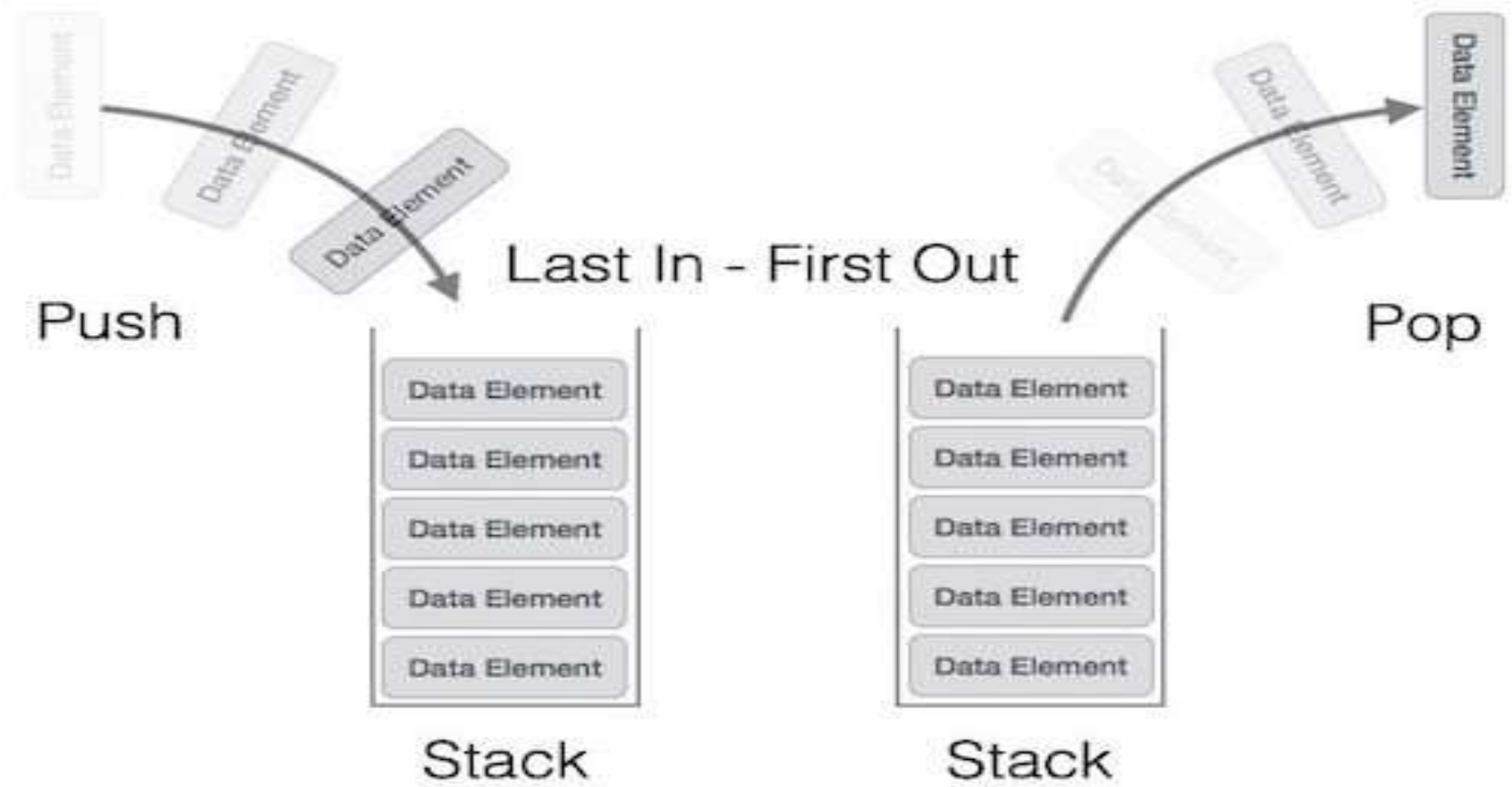
A real-world stack allows operations at one end only. For example, we can place or remove a card or plate from the top of the stack only. Likewise, Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack.

This feature makes it LIFO data structure. LIFO stands for Last-in-first-out. Here, the element which is placed (inserted or added) last, is accessed first. In stack terminology, insertion operation is called PUSH operation and removal operation is called POP operation.

Stack..

Stack Representation:

The following diagram depicts a stack and its operations −



Stack..

A stack can be implemented by means of Array, Structure, Pointer, and Linked List. Stack can either be a fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack using arrays, which makes it a fixed size stack implementation.

Stack Opertaion

Basic Operations:

Stack operations may involve initializing the stack, using it and then de-initializing it. Apart from these basic stuffs, a stack is used for the following two primary operations −

- push() − Pushing (storing) an element on the stack.
- pop() − Removing (accessing) an element from the stack.

When data is PUSHed onto stack.

To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks −

- peek() − get the top data element of the stack, without removing it.
- isFull() − check if stack is full.
- isEmpty() − check if stack is empty.

Stack Opertaion..

At all times, we maintain a pointer to the last PUSHed data on the stack. As this pointer always represents the top of the stack, hence named top. The top pointer provides top value of the stack without actually removing it.

First we should learn about procedures to support stack functions −

peek()

Algorithm of peek() function −

```
begin procedure peek
        Return stack[top]
end procedure
```

Stack Opertaion..

Implementation of peek() function in C programming language −

```
int peek() {
return stack[top];
}
```

Stack Opertaion..

- **isfull()**

Algorithm of isfull() function −

```
begin procedure isfull
   if top equals to MAXSIZE
      return true
   else
      return false
   endif
end procedure
```

Stack Opertaion..

Implementation of isfull() function in C programming language −

```
bool isfull() {
    if(top == MAXSIZE)
      return true;
    else
      return false;
}
```

Stack Opertaion..

- **isempty()**

Algorithm of isempty() function −

```
begin procedure isempty
   if top less than 1
      return true
   else
      return false
   endif
end procedure
```

Stack Opertaion..

Implementation of isempty() function in C programming language is slightly different. We initialize top at -1, as the index in array starts from 0. So we check if the top is below zero or -1 to determine if the stack is empty. Here's the code −

```
bool isempty() {
   if(top == -1)
      return true;
   else
      return false;
}
```

Stack Opertaion..

Push Operation:

The process of putting a new data element onto stack is known as a Push Operation. Push operation involves a series of steps −

Step 1 − Checks if the stack is full.

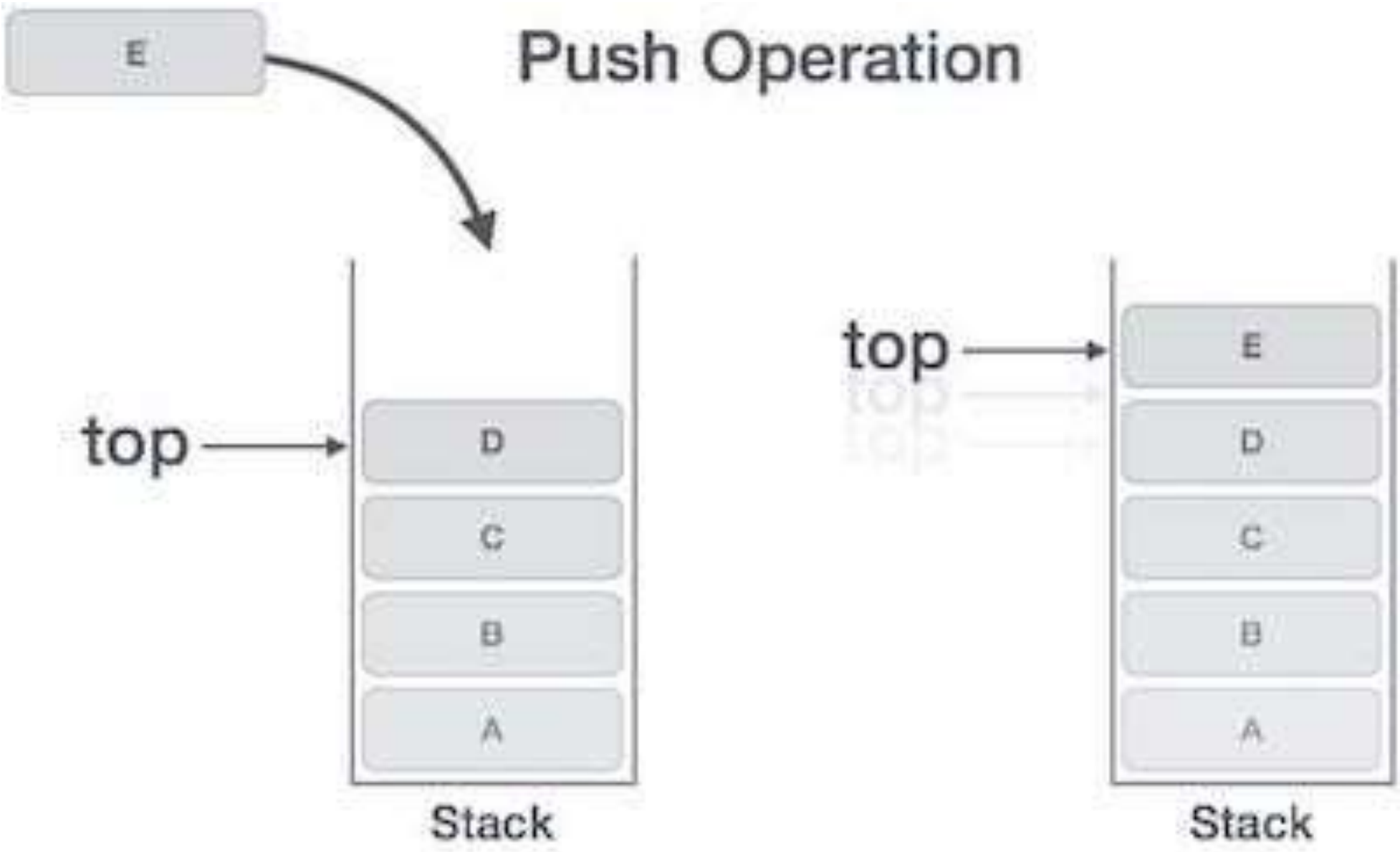
Step 2 − If the stack is full, produces an error and exit.

Step 3 − If the stack is not full, increments top to point next empty space.

Step 4 − Adds data element to the stack location, where top is pointing.

Step 5 − Returns success.

Stack Opertaion..



Stack Opertaion..

Algorithm for PUSH Operation:

```
begin procedure push: stack, data
   if stack is full
      return null
   endif
   top ← top + 1
   stack[top] ← data
end procedure
```

Stack Opertaion..

Implementation of this algorithm in C, is very easy. See the following code −

```
void push(int data) {
if(!isFull()) {
      top = top + 1;
      stack[top] = data;
   } else {
      printf("Could not insert data, Stack is full.\n");
   }
}
```

Stack Opertaion..

Pop Operation:

Accessing the content while removing it from the stack, is known as a Pop Operation. In an array implementation of pop() operation, the data element is not actually removed, instead top is decremented to a lower position in the stack to point to the next value. But in linked-list implementation, pop() actually removes data element and deallocates memory space.

Stack Opertaion..

A Pop operation may involve the following steps −

Step 1 − Checks if the stack is empty.

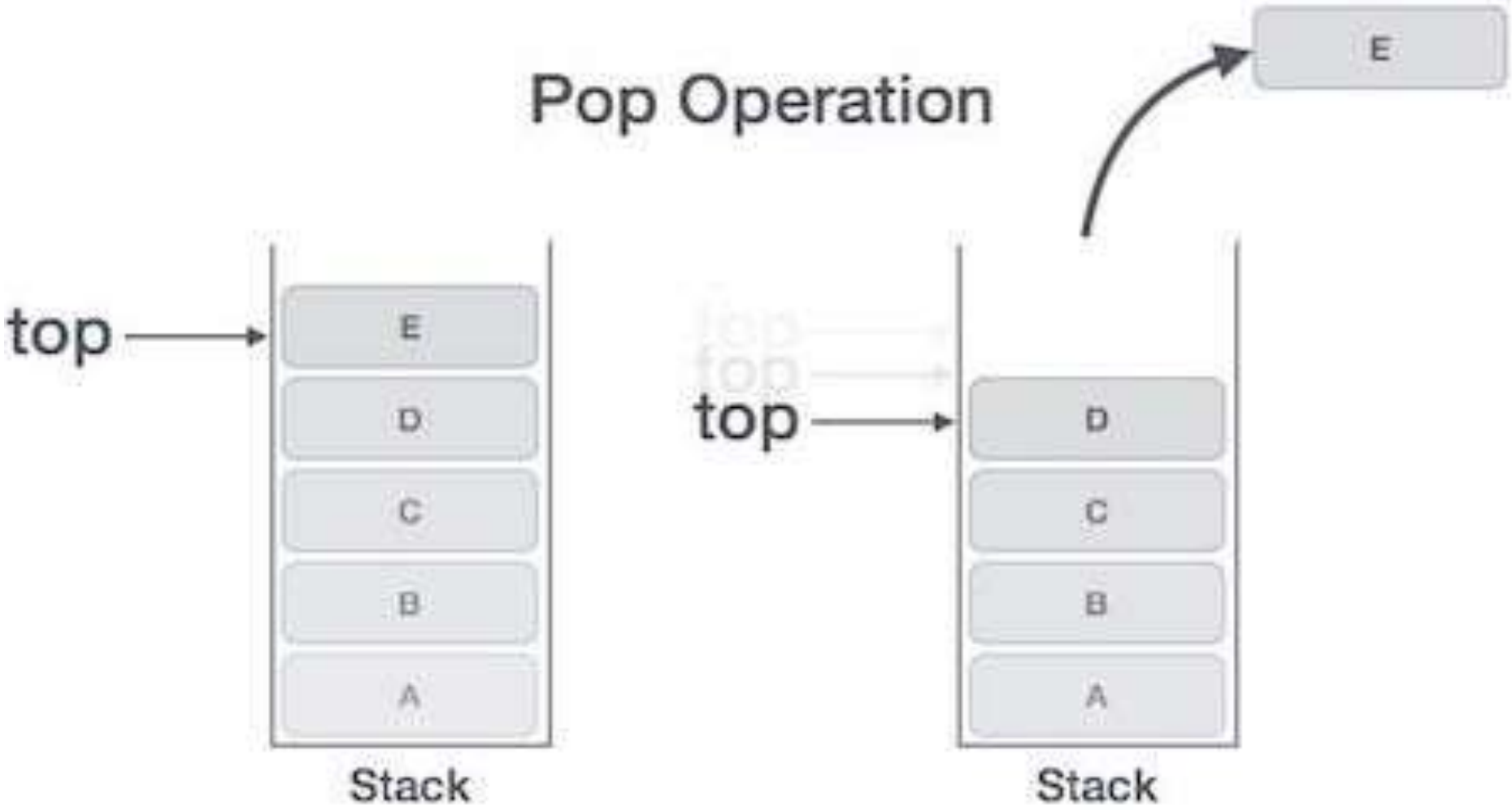
Step 2 − If the stack is empty, produces an error and exit.

Step 3 − If the stack is not empty, accesses the data element at which top is pointing.

Step 4 − Decreases the value of top by 1.

Step 5 − Returns success.

Stack Opertaion..



Stack Opertaion..

Algorithm for Pop Operation:

```
begin procedure pop: stack

   if stack is empty
      return null
   endif

   data ← stack[top]
   top ← top - 1
   return data

end procedure
```

Stack Opertaion..

Implementation of this algorithm in C, is as follows −

```
int pop(int data) {

   if(!isempty()) {
      data = stack[top];
      top = top - 1;
      return data;
   } else {
      printf("Could not retrieve data, Stack is empty.\n");
   }
}
```

Stack Application

The way to write arithmetic expression is known as a notation. An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression. These notations are −

Infix Notation

Prefix (Polish) Notation

Postfix (Reverse-Polish) Notation

Stack Application..

Infix Notation:

We write expression in infix notation, e.g. a - b + c, where operators are used in-between operands.

Prefix Notation:

In this notation, operator is prefixed to operands, i.e. operator is written ahead of operands. For example, +ab. This is equivalent to its infix notation a + b. Prefix notation is also known as Polish Notation.

Postfix Notation:

This notation style is known as Reversed Polish Notation. In this notation style, the operator is postfixed to the operands i.e., the operator is written after the operands. For example, ab+. This is equivalent to its infix notation a + b.

Stack Application..

Sr.No.	Infix Notation	Prefix Notation	Postfix Notation
1	a + b	+ a b	a b +
2	(a + b) ∗ c	∗ + a b c	a b + c ∗
3	a ∗ (b + c)	∗ a + b c	a b c + ∗
4	a / b + c / d	+ / a b / c d	a b / c d / +
5	(a + b) ∗ (c + d)	∗ + a b + c d	a b + c d + ∗
6	((a + b) ∗ c) - d	- ∗ + a b c d	a b + c ∗ d -

Stack Application..

Parsing Expressions:

It is not a very efficient way to design an algorithm or program to parse infix notations. Instead, these infix notations are first converted into either postfix or prefix notations and then computed.

To parse any arithmetic expression, we need to take care of operator precedence and associativity also.

Precedence:

When an operand is in between two different operators, which operator will take the operand first, is decided by the precedence of an operator over others. For example −

$$a + b * c \Rightarrow a + (b * c)$$

Operator Precendence

As multiplication operation has precedence over addition, b * c will be evaluated first.

Stack Application..

Associativity:

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression a + b − c, both + and – have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators. Here, both + and − are left associative, so the expression will be evaluated as (a + b) − c.

Stack Application..

Precedence and associativity determines the order of evaluation of an expression. Following is an operator precedence and associativity table (highest to lowest) −

Sr.No	Operator	Precedence	Associativity
1	Exponentiation ^	Highest	Right Associative
2	Multiplication (∗) & Division (/)	Second Highest	Left Associative
3	Addition (+) & Subtraction (−)	Lowest	Left Associative

Stack Application..

The above table shows the default behavior of operators. At any point of time in expression evaluation, the order can be altered by using parenthesis. For example −

In a + b*c, the expression part b*c will be evaluated first, with multiplication as precedence over addition. We here use parenthesis for a + b to be evaluated first, like (a + b)*c.

Postfix Evaluation using Stack

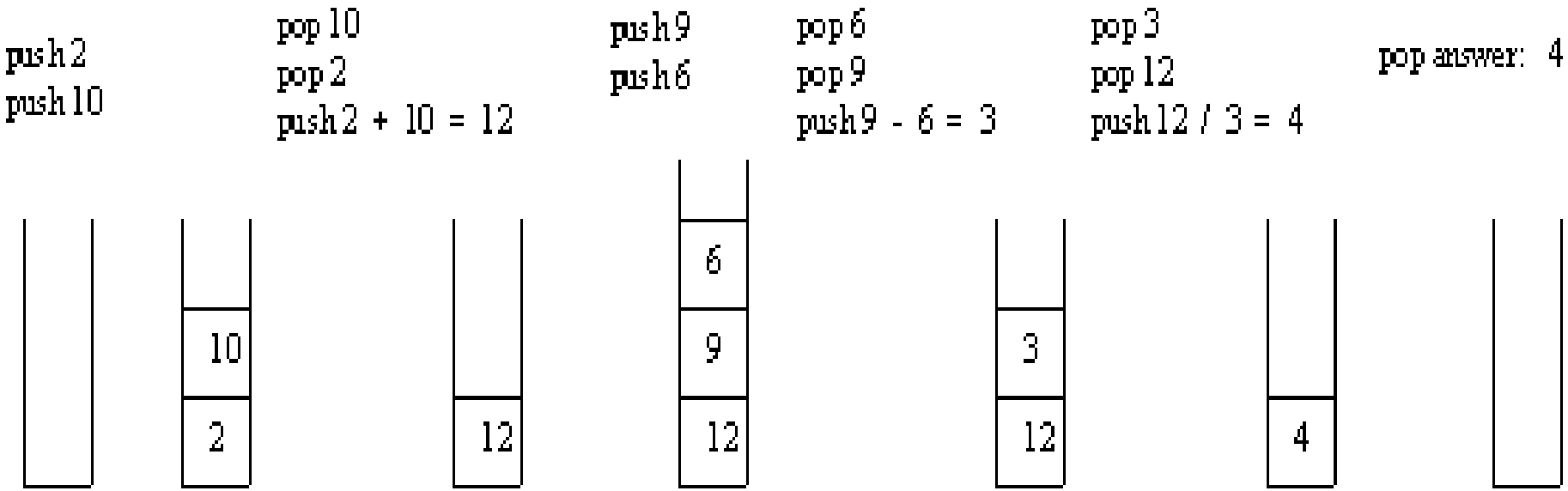
Postfix Evaluation Algorithm:

Step 1 – scan the expression from left to right
Step 2 – if it is an operand push it to stack
Step 3 – if it is an operator pull operand from stack and perform operation .
Step 4 – store the output of step 3, back to stack
Step 5 – scan the expression until all operands are consumed
Step 6 – pop the stack and perform operation

Postfix Evaluation using Stack..

Example:

2 10 + 9 6 - /



Infix To Postfix Conversion Using Stack

One of the applications of Stack is in the conversion of arithmetic expressions in high-level programming languages into machine readable form. As our computer system can only understand and work on a binary language, it assumes that an arithmetic operation can take place in two operands only e.g., A+B, C*D,D/A etc. But in our usual form an arithmetic expression may consist of more than one operator and two operands e.g. (A+B)*C(D/(J+D)).

These complex arithmetic operations can be converted into reverse polish notation using stacks which then can be executed in two operands and an operator form.

Algorithm to convert Infix To Postfix:

Let, X is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression Y.

1. Push "("onto Stack, and add ")" to the end of X.
2. Scan X from left to right and repeat Step 3 to 6 for each element of X until the Stack is empty.
3. If an operand is encountered, add it to Y.
4. If a left parenthesis is encountered, push it onto Stack.
5. If an operator is encountered ,then:

(a) Repeatedly pop from Stack and add to Y each operator (on the top of Stack) which has the same precedence as or higher precedence than operator.

(b) Add operator to Stack.

[End of If]

6. If a right parenthesis is encountered ,then:

(a) Repeatedly pop from Stack and add to Y each operator (on the top of Stack) until a left parenthesis is encountered.

(b) Remove the left Parenthesis.

[End of If]

[End of If]

7. END.

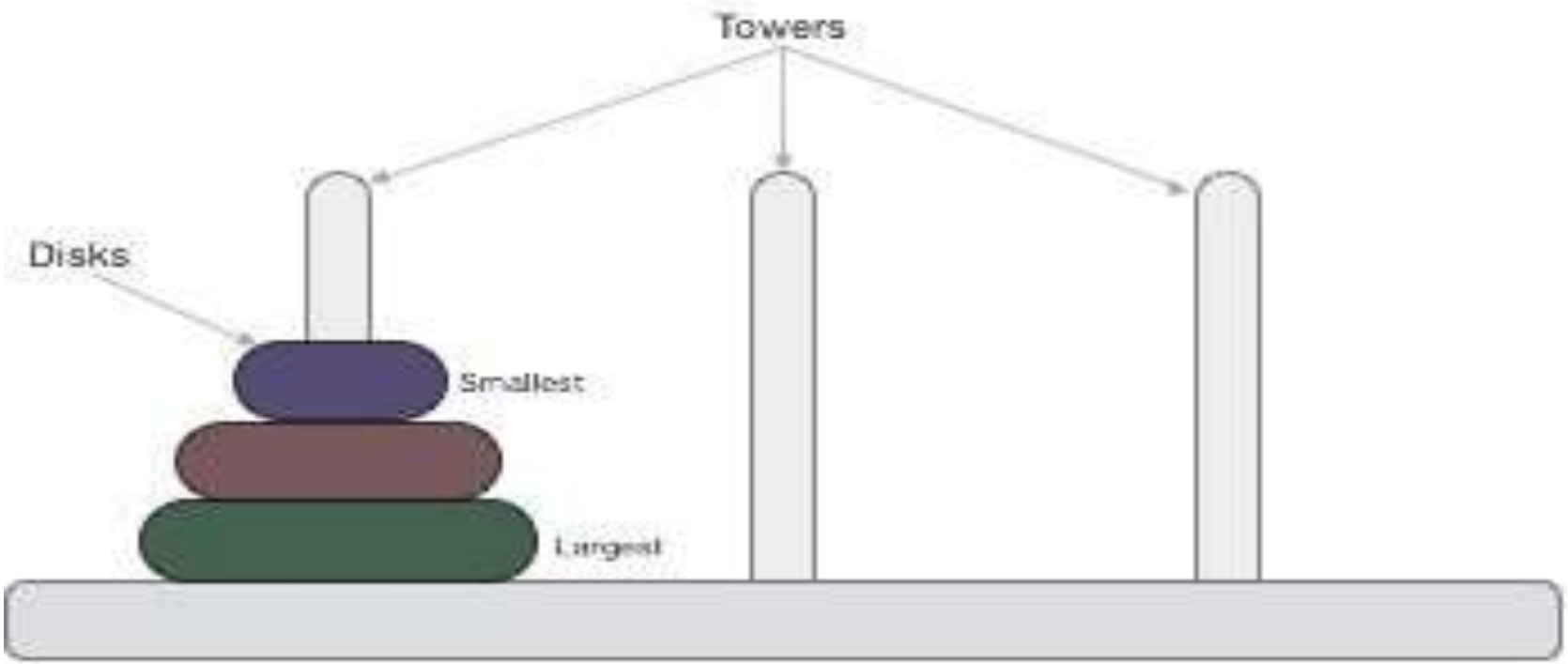
Example

Infix Expression: A+ (B*C-(D/E^F)*G)*H, where ^ is an exponential operator.

Symbol	Scanned	STACK	Postfix Expression	Description
1.		(		Start
2.	A	(	A	
3.	+	(+	A	
4.	(	(+(	A	
5.	B	(+(	AB	
6.	*	(+(*	AB	
7.	C	(+(*	ABC	
8.	-	(+(-	ABC*	'*' is at higher precedence than '-'
9.	(	(+(-(	ABC*	
10.	D	(+(-(	ABC*D	
11.	/	(+(-(/	ABC*D	
12.	E	(+(-(/	ABC*DE	
13.	^	(+(-(/^	ABC*DE	
14.	F	(+(-(/^	ABC*DEF	
15.	)	(+(-	ABC*DEF^/	Pop from top on Stack, that's why '^' Come first
16.	*	(+(-*	ABC*DEF^/	
17.	G	(+(-*	ABC*DEF^/G	
18.	)	(+	ABC*DEF^/G*-	Pop from top on Stack, that's why '^' Come first
19.	*	(+*	ABC*DEF^/G*-	
20.	H	(+*	ABC*DEF^/G*-H	
21.	)	Empty	ABC*DEF^/G*-H*+	END

Tower of Hanoi

Tower of Hanoi, is a mathematical puzzle which consists of three towers (pegs) and more than one rings is as depicted −



Tower of Hanoi..

These rings are of different sizes and stacked upon in an ascending order, i.e. the smaller one sits over the larger one. There are other variations of the puzzle where the number of disks increase, but the tower count remains the same.

Tower of Hanoi..

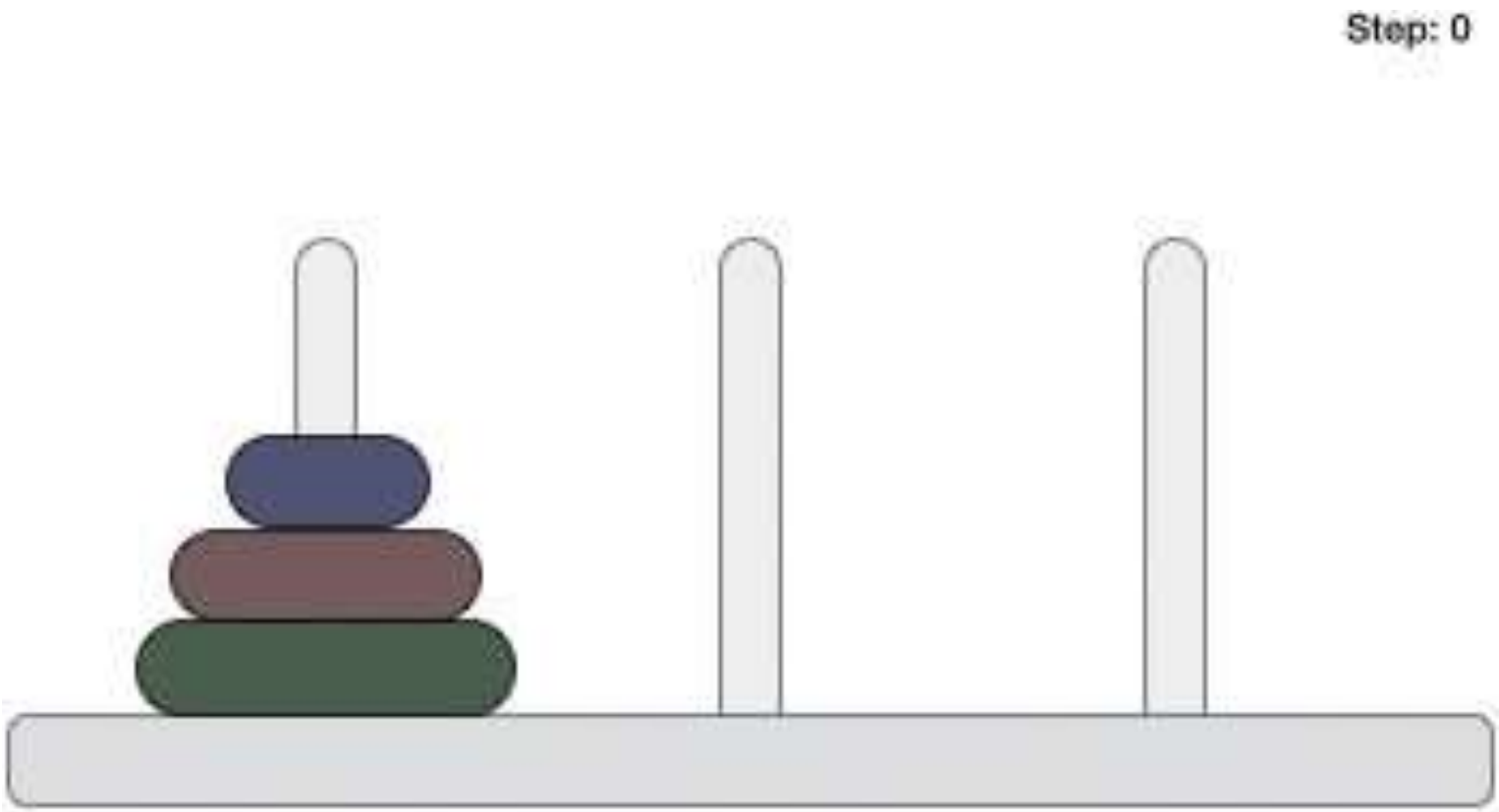
Rules:

The mission is to move all the disks to some another tower without violating the sequence of arrangement. A few rules to be followed for Tower of Hanoi are −

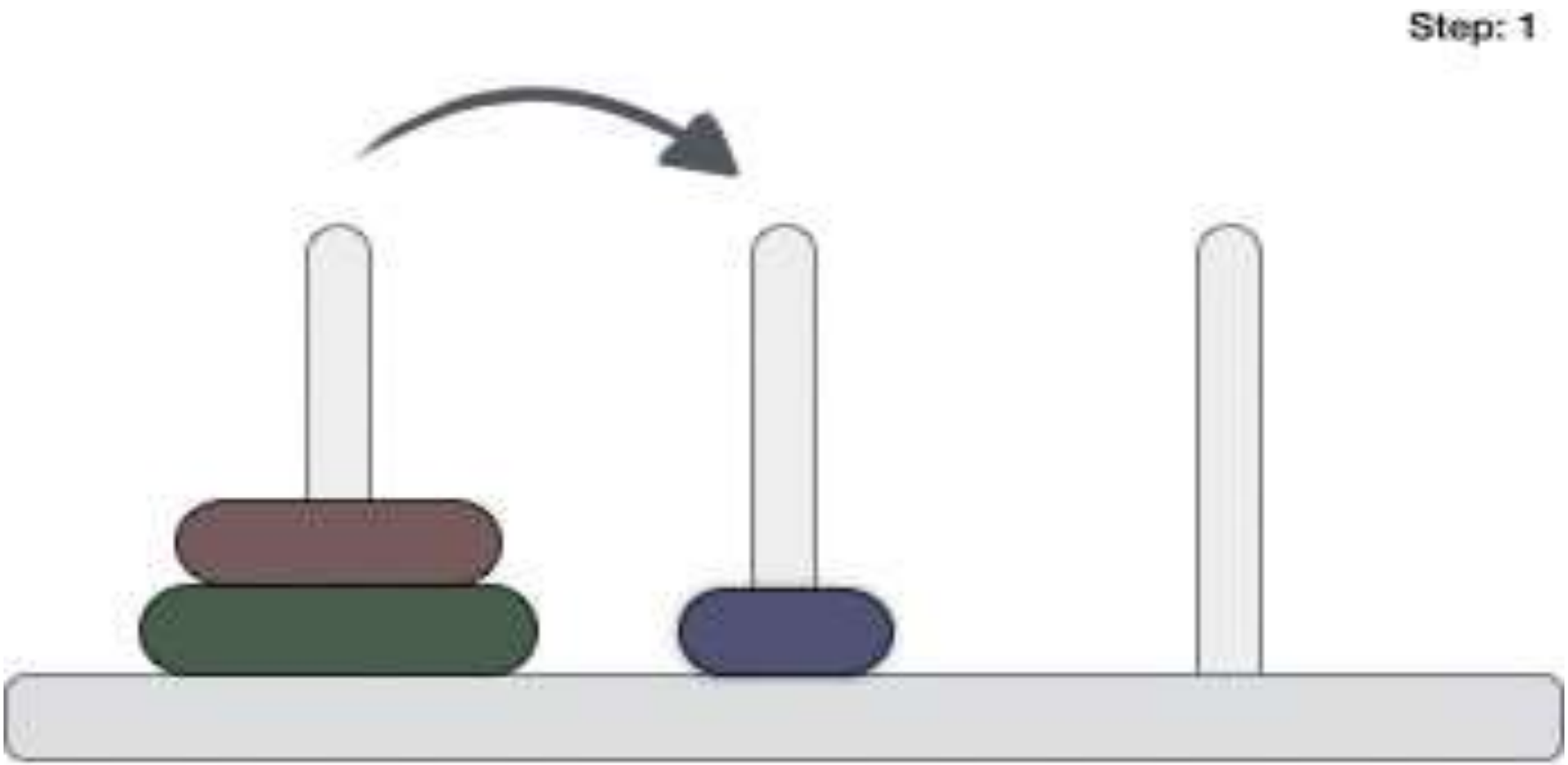
- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.
- No large disk can sit over a small disk.

Tower of Hanoi..

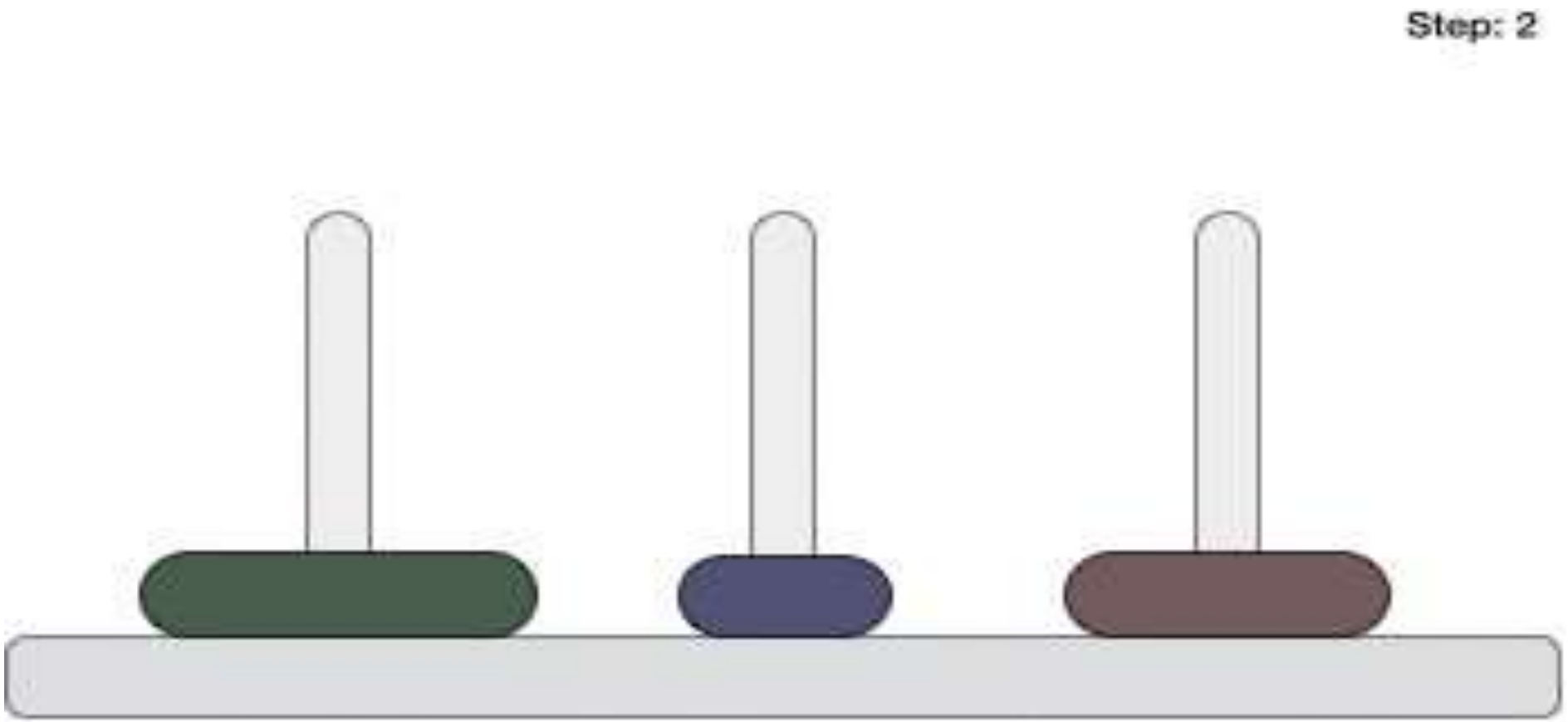
Following is an representation of solving a Tower of Hanoi puzzle with three disks.



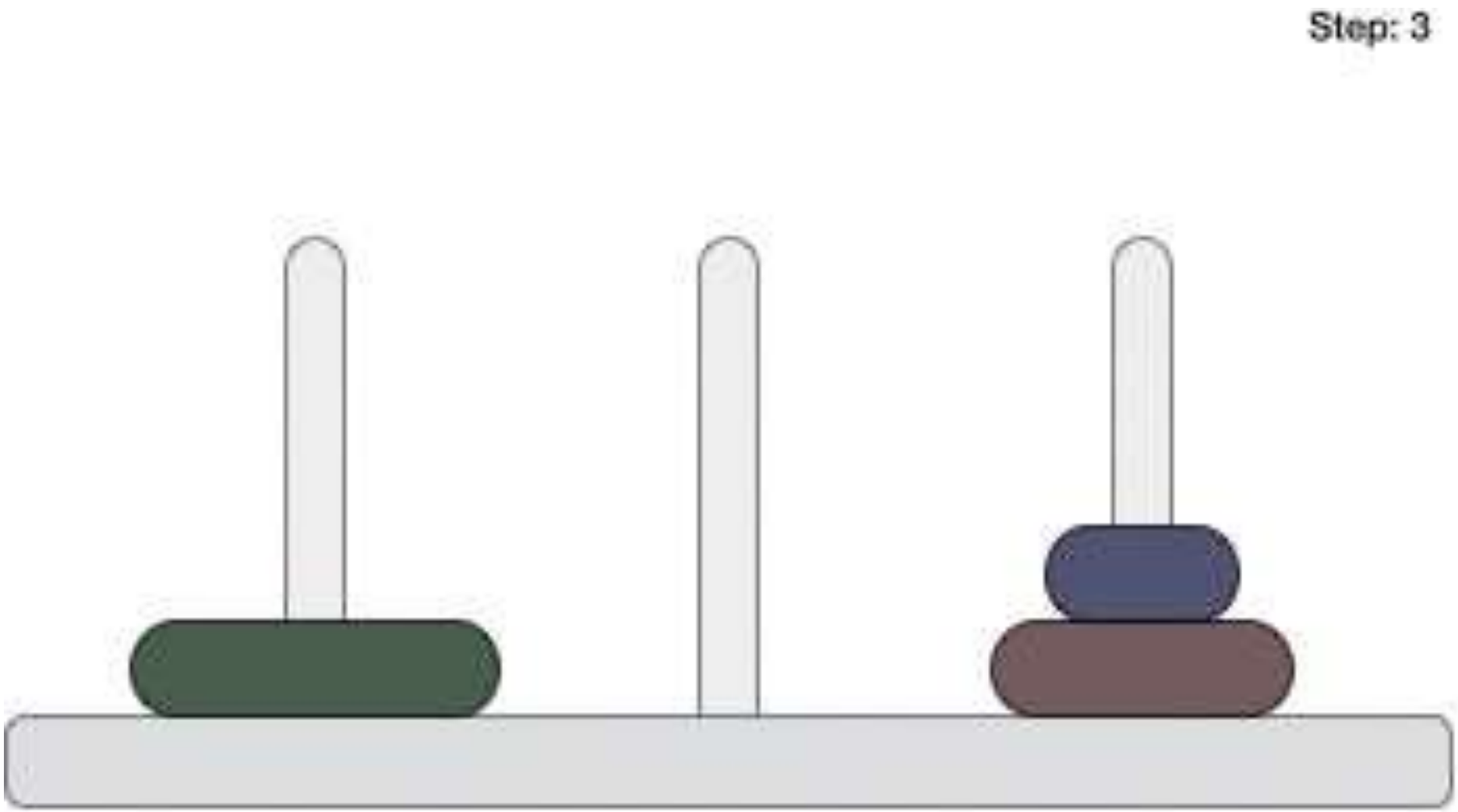
Tower of Hanoi..



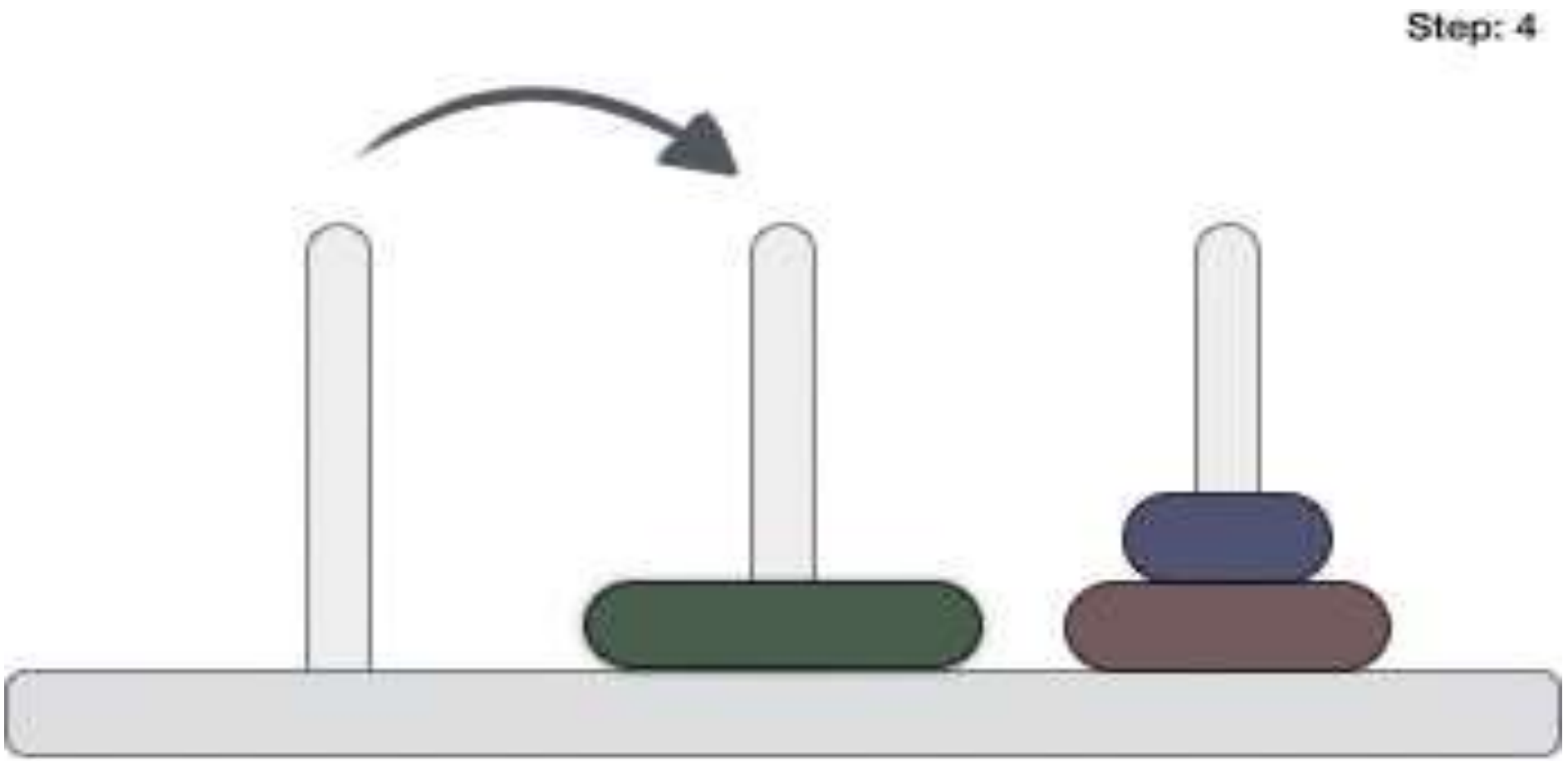
Tower of Hanoi..



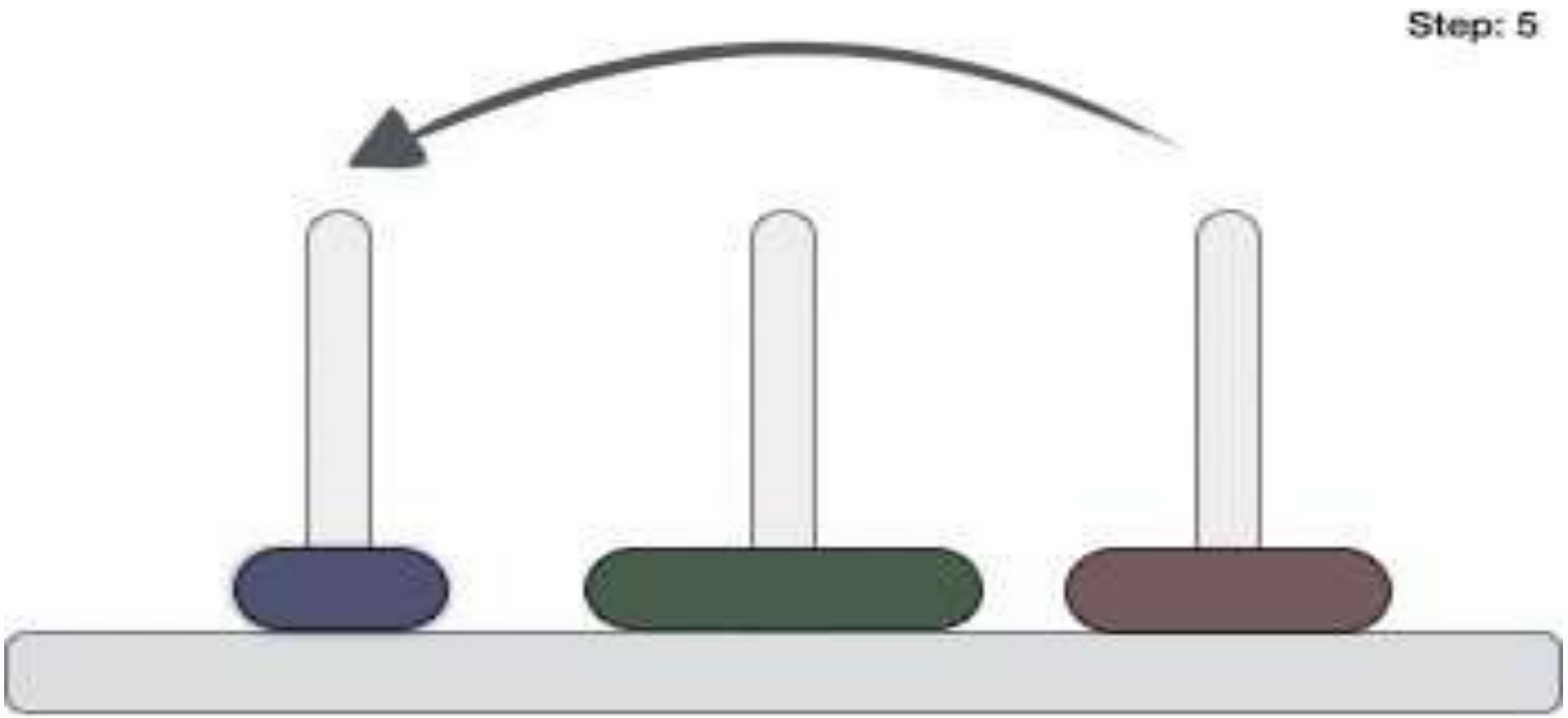
Tower of Hanoi..



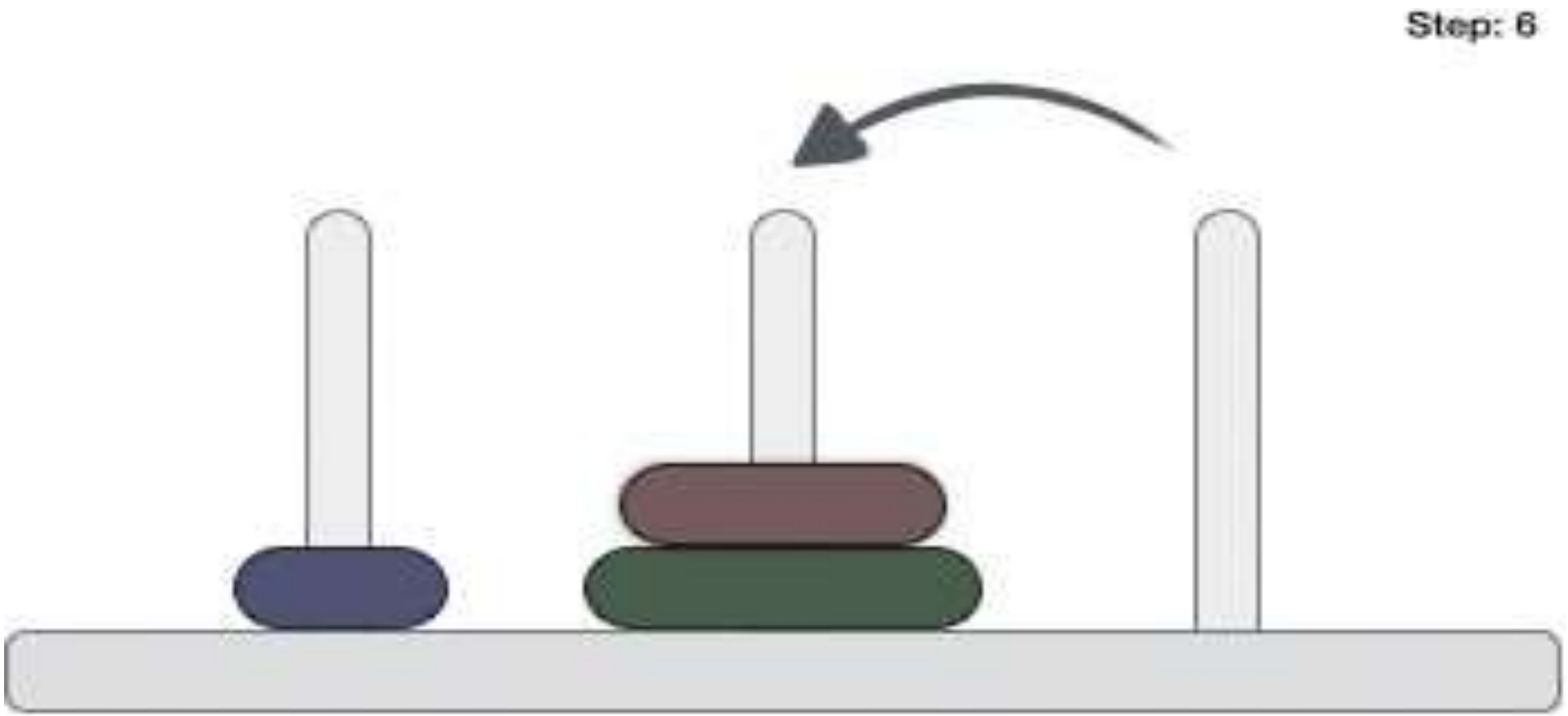
Tower of Hanoi..



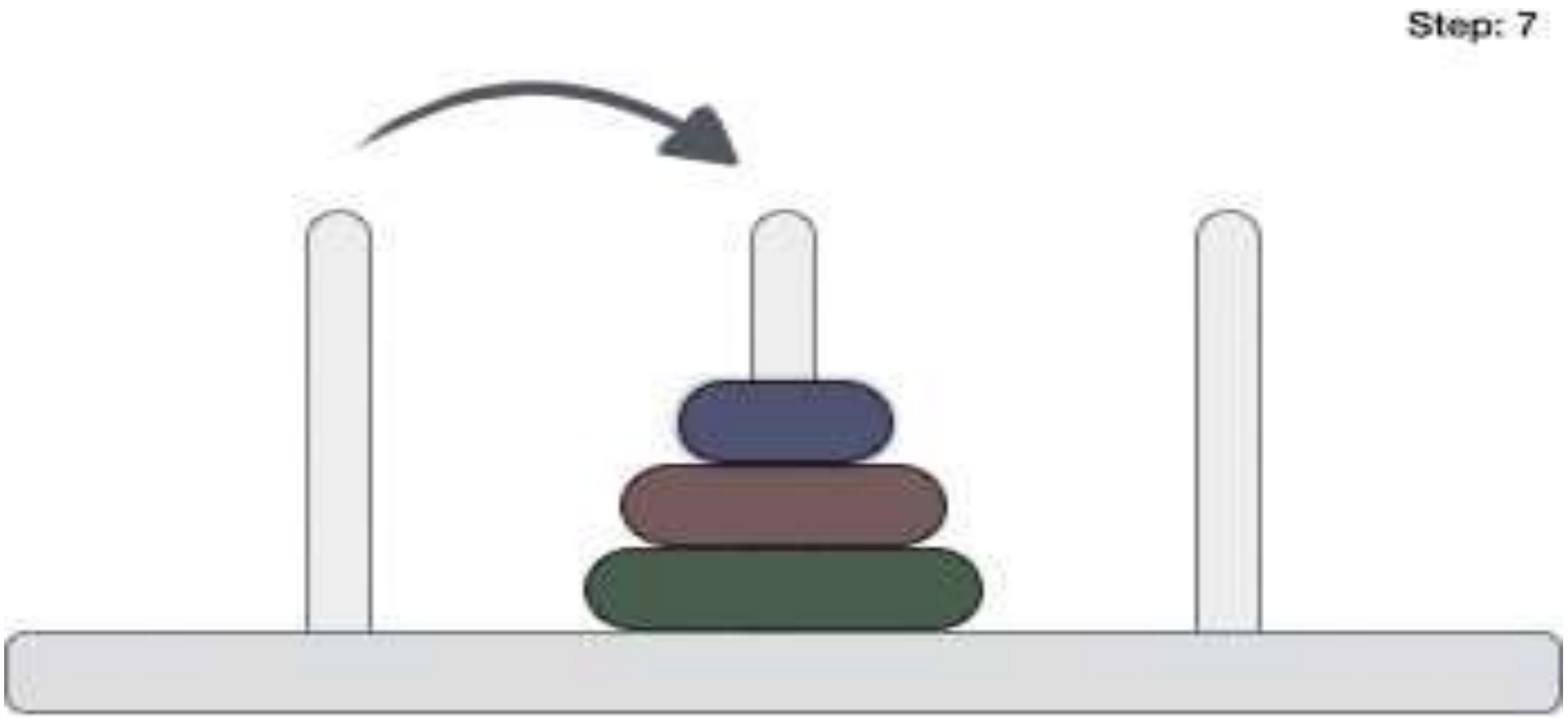
Tower of Hanoi..



Tower of Hanoi..



Tower of Hanoi..



Tower of Hanoi..

Tower of Hanoi puzzle with n disks can be solved in minimum 2^n−1 steps. This presentation shows that a puzzle with 3 disks has taken 2^3 - 1 = 7 steps.

Tower of Hanoi..

Algorithm:

To write an algorithm for Tower of Hanoi, first we need to learn how to solve this problem with lesser amount of disks, say → 1 or 2. We mark three towers with name, source, destination and aux (only to help moving the disks). If we have only one disk, then it can easily be moved from source to destination peg.

If we have 2 disks −

First, we move the smaller (top) disk to aux peg.

Then, we move the larger (bottom) disk to destination peg.

And finally, we move the smaller disk from aux to destination peg.

Tower of Hanoi..

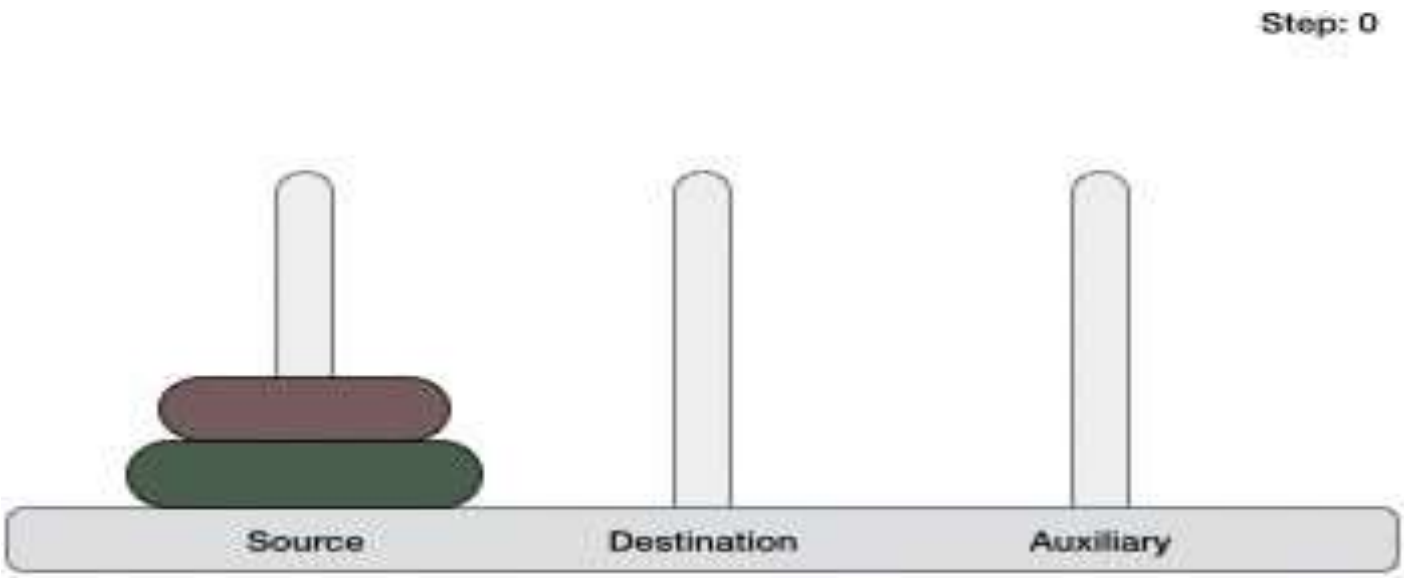
So now, we are in a position to design an algorithm for Tower of Hanoi with more than two disks. We divide the stack of disks in two parts. The largest disk (nth disk) is in one part and all other (n-1) disks are in the second part.

Our ultimate aim is to move disk n from source to destination and then put all other (n1) disks onto it. We can imagine to apply the same in a recursive way for all given set of disks.

Tower of Hanoi..

The steps to follow are −

Step 1 − Move n-1 disks from source to aux
Step 2 − Move nth disk from source to dest
Step 3 − Move n-1 disks from aux to dest



Tower of Hanoi..

A recursive algorithm for Tower of Hanoi can be driven as follows −

```
START
Procedure Hanoi(disk, source, dest, aux)

   IF disk == 1, THEN
      move disk from source to dest
   ELSE
      Hanoi(disk - 1, source, aux, dest)     // Step 1
      move disk from source to dest          // Step 2
      Hanoi(disk - 1, aux, dest, source)     // Step 3
   END IF

END Procedure
STOP
```

Reverse a String using STACK

This algorithm will read a string and reverse the string using Stack push and pop operations.

Reversing string is an operation of Stack by using Stack we can reverse any string.

Algorithm:

1. Read a string.
2. Push all characters until NULL is not found - Characters will be stored in stack variable.
3. Pop all characters until NULL is not found - As we know stack is a LIFO technique, so last character will be pushed first and finally we will get reversed string in a variable in which we store inputted string.

Factorial using stack

Factorial of a positive integer is the product of an integer and all the integers below it, i.e., the factorial of number n (represented by n!) would be given by

$$n! = 1*2*3*4* \ldots\ldots *n$$

The factorial of 0 is defined to be 1 and is not defined for negative integers. There are multiple ways to find it which are listed below-

- Factorial Program in C using For loop
- Factorial Program using Functions
- Factorial Program using Recursion
- Factorial Program using Stack

Topics to be covered
Stack

What do you mean by Queue ?

Opertaions on Queue

What do you mean by Queue ?

Queue is an abstract data structure, somewhat similar to Stacks. Unlike stacks, a queue is open at both its ends. One end is always used to insert data (enqueue) and the other is used to remove data (dequeue). Queue follows First-In-First-Out methodology, i.e., the data item stored first will be accessed first.

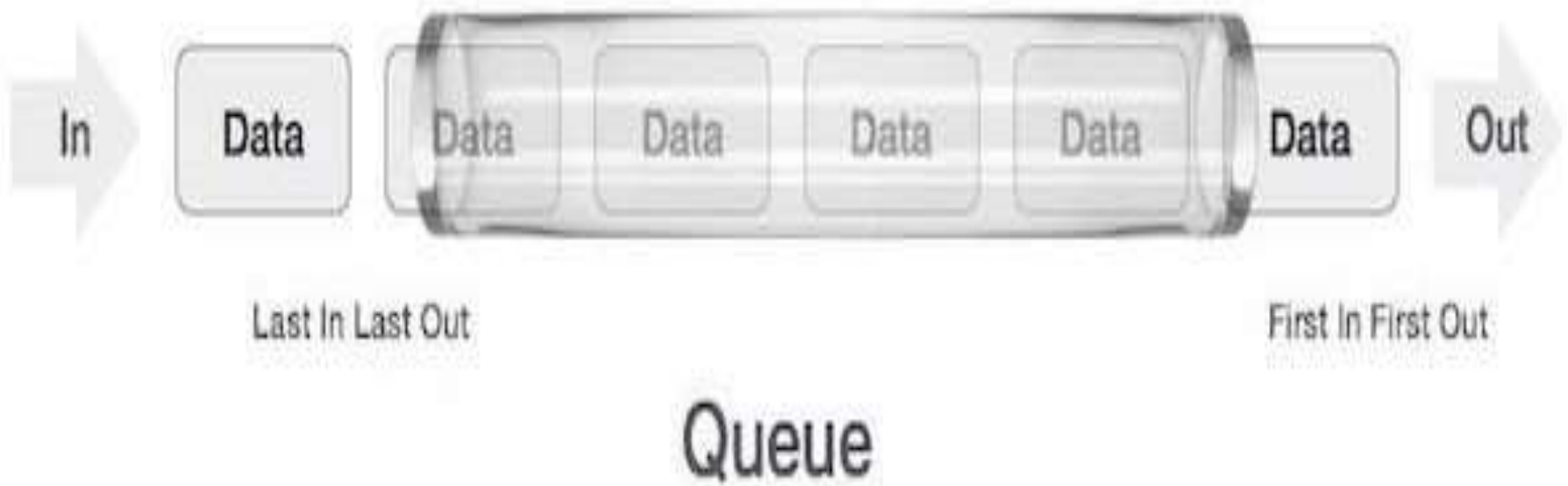
Queue..

A real-world example of queue can be a single-lane one-way road, where the vehicle enters first, exits first. More real-world examples can be seen as queues at the ticket windows and bus-stops.

Queue..

Queue Representation:

As we now understand that in queue, we access both ends for different reasons. The following diagram given below tries to explain queue representation as data structure −



Queue..

As in stacks, a queue can also be implemented using Arrays, Linked-lists, Pointers and Structures. For the sake of simplicity, we shall implement queues using one-dimensional array.

Basic Operations

Basic Operations:

Queue operations may involve initializing or defining the queue, utilizing it, and then completely erasing it from the memory. Here we shall try to understand the basic operations associated with queues −

- **Insertion** − add (enqueue) an item to the queue.
- **Deletion** − remove (dequeue) an item from the queue.

Basic Operations..

Few more functions are required to make the above-mentioned queue operation efficient. These are −

- **peek()** − Gets the element at the front of the queue without removing it.
- **isfull()** − Checks if the queue is full.
- **isempty()** − Checks if the queue is empty.

In queue, we always dequeue (or access) data, pointed by front pointer and while enqueing (or storing) data in the queue we take help of rear pointer.

Basic Operations..

First learn about supportive functions of a queue −

Peek():

This function helps to see the data at the front of the queue. The algorithm of peek() function is as follows −

```
begin procedure peek
   return queue[front]
end procedure
```

Basic Operations..

Implementation of peek() function in C programming language −

```
int peek() {
   return queue[front];
}
```

Basic Operations..

Isfull():

As we are using single dimension array to implement queue, we just check for the rear pointer to reach at MAXSIZE to determine that the queue is full. In case we maintain the queue in a circular linked-list, the algorithm will differ. Algorithm of isfull() function −

```
begin procedure isfull

   if rear equals to MAXSIZE
      return true
   else
      return false
   endif

end procedure
```

Basic Operations..

Implementation of isfull() function in C programming language −

```
bool isfull() {
   if(rear == MAXSIZE - 1)
      return true;
   else
      return false;
}
```

Basic Operations..

Isempty():

Algorithm of isempty() function −

```
begin procedure isempty

   if front is less than MIN  OR front is greater than rear
      return true
   else
      return false
   endif

end procedure
```

Basic Operations..

If the value of front is less than MIN or 0, it tells that the queue is not yet initialized, hence empty.

Here's the C programming code −

```
bool isempty() {
   if(front < 0 || front > rear)
      return true;
   else
      return false;
}
```

Basic Operations..

Insertion Operation:

Queues maintain two data pointers, front and rear. Therefore, its operations are comparatively difficult to implement than that of stacks.

The following steps should be taken to enqueue (insert) data into a queue −

Step 1 − Check if the queue is full.

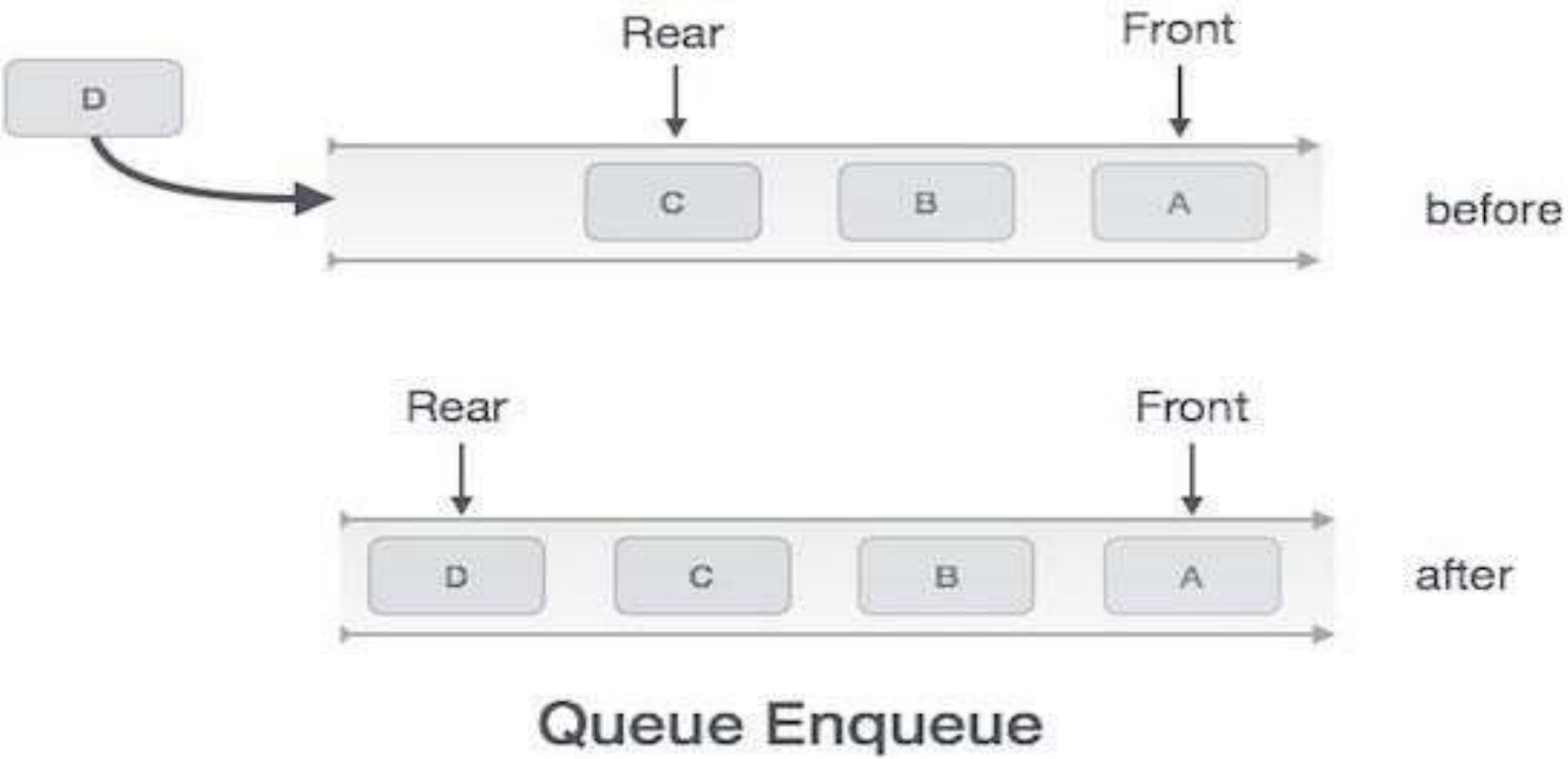
Step 2 − If the queue is full, produce overflow error and exit.

Step 3 − If the queue is not full, increment rear pointer to point the next empty space.

Step 4 − Add data element to the queue location, where the rear is pointing.

Step 5 − return success.

Basic Operations..



Basic Operations..

Algorithm for enqueue(Insertion) operation:

```
procedure enqueue(data)

   if queue is full
      return overflow
   endif

   rear ← rear + 1
   queue[rear] ← data
   return true

end procedure
```

Basic Operations..

Implementation of enqueue() in C programming language −

```
int enqueue(int data)
   if(isfull())
      return 0;

   rear = rear + 1;
   queue[rear] = data;

   return 1;
end procedure
```

Basic Operations..

Deletion Operation:

Accessing data from the queue is a process of two tasks − access the data where front is pointing and remove the data after access. The following steps are taken to perform dequeue operation −

Step 1 − Check if the queue is empty.

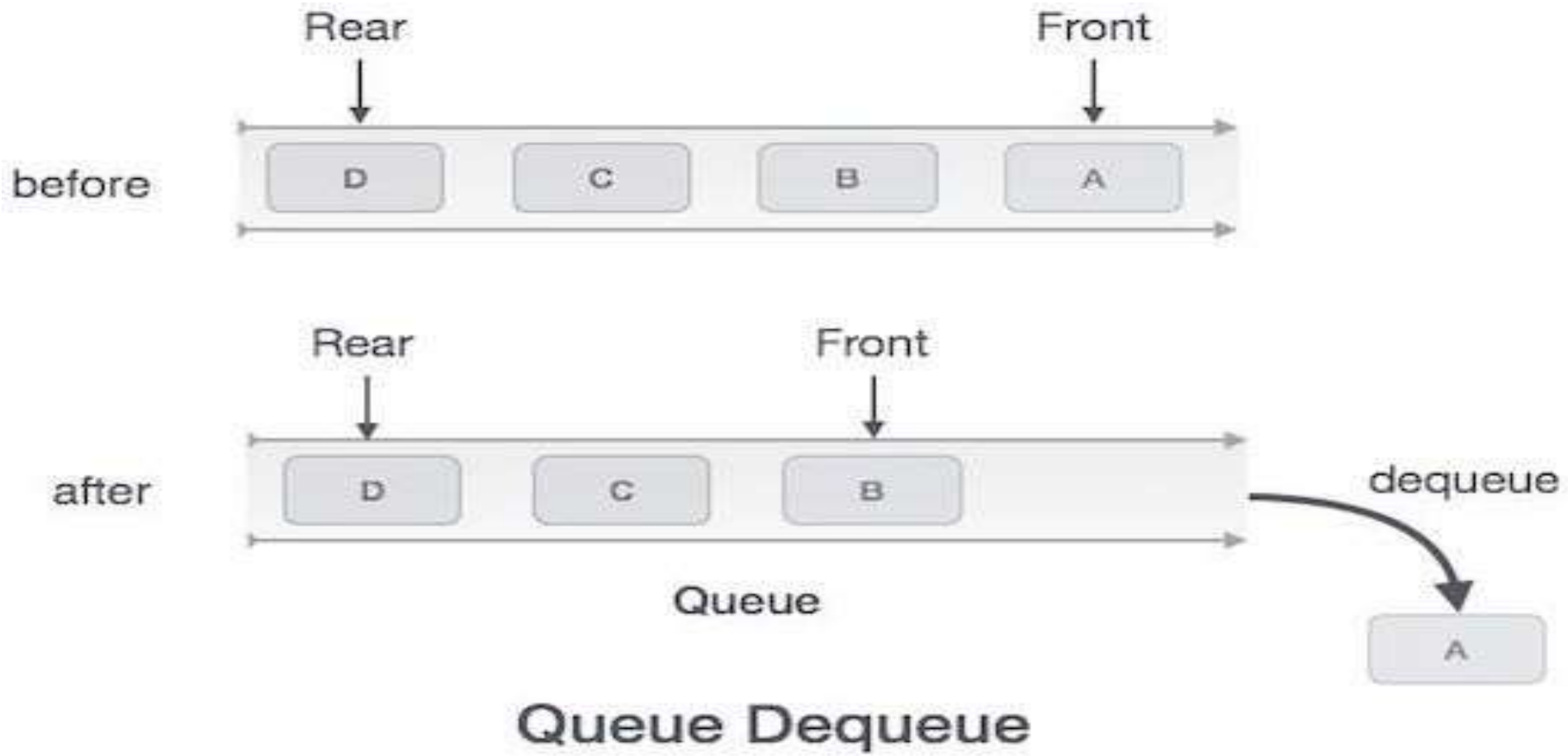
Step 2 − If the queue is empty, produce underflow error and exit.

Step 3 − If the queue is not empty, access the data where front is pointing.

Step 4 − Increment front pointer to point to the next available data element.

Step 5 − Return success.

Basic Operations..



Queue Dequeue

Basic Operations..

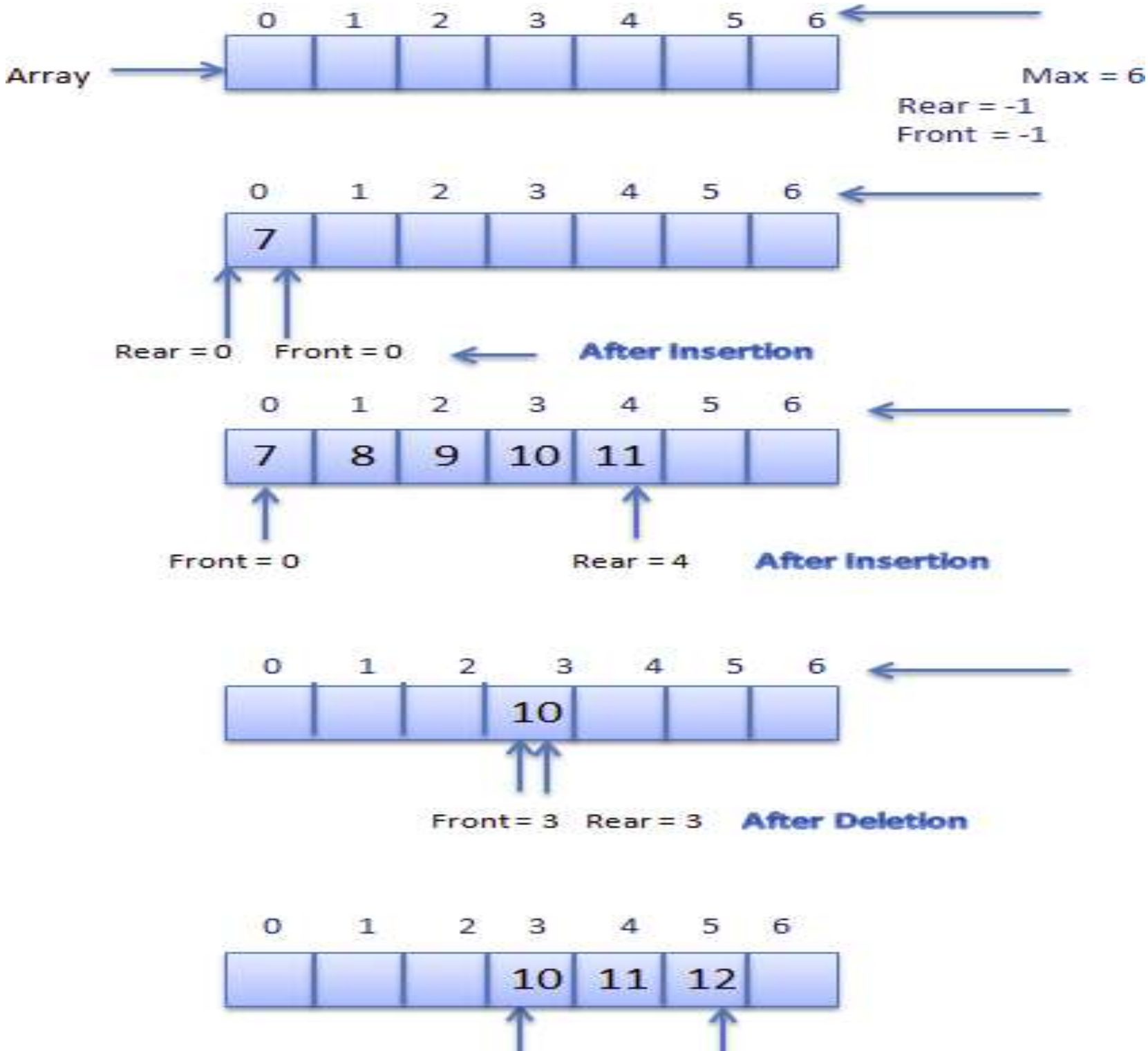
Algorithm for dequeue operation:

```
procedure dequeue

   if queue is empty
      return underflow
   end if

   data = queue[front]
   front ← front + 1
   return true

end procedure
```

Basic Operations..

Implementation of dequeue() in C programming language −

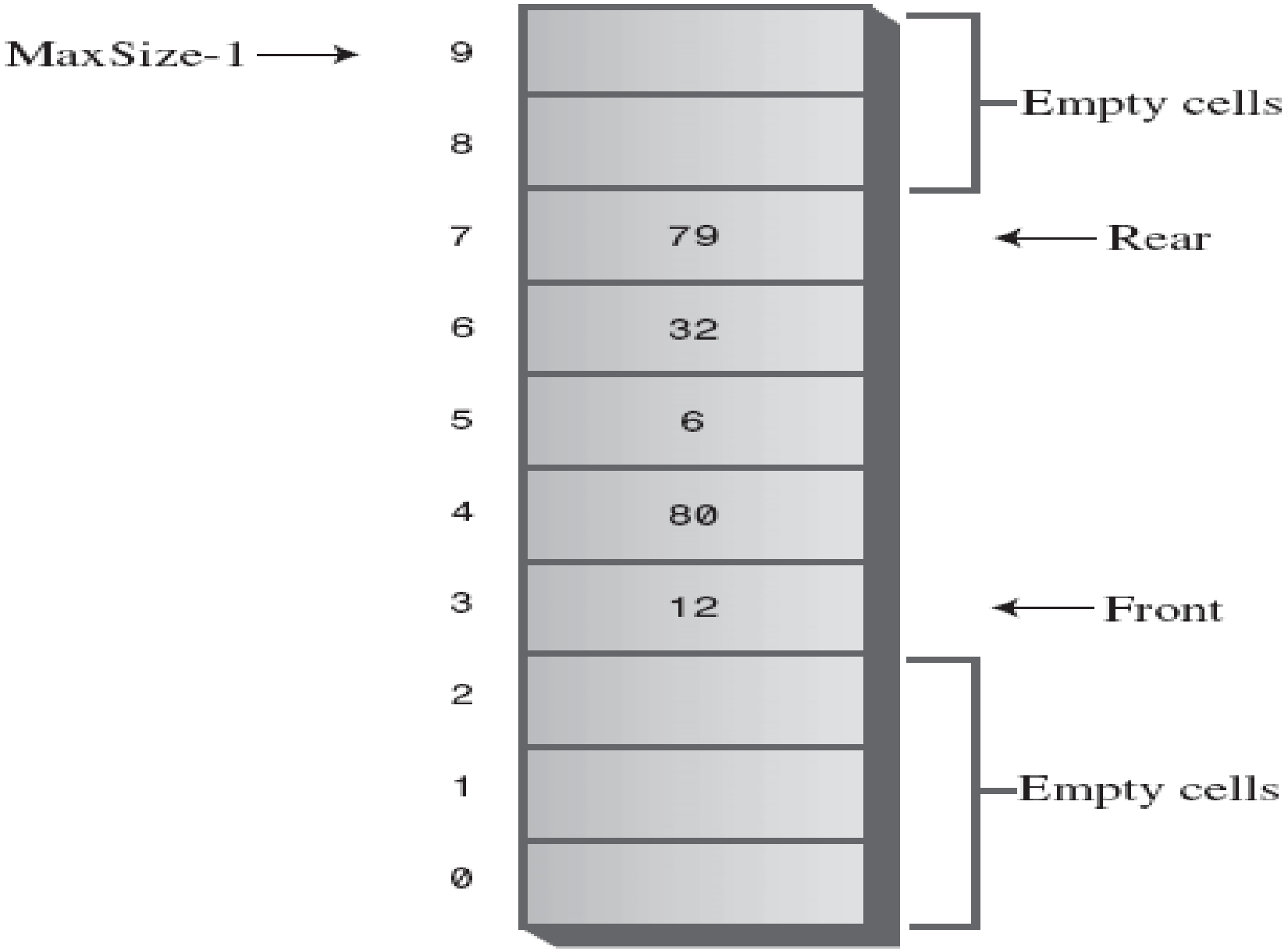
```
int dequeue() {
   if(isempty())
      return 0;

   int data = queue[front];
   front = front + 1;

   return data;
}
```

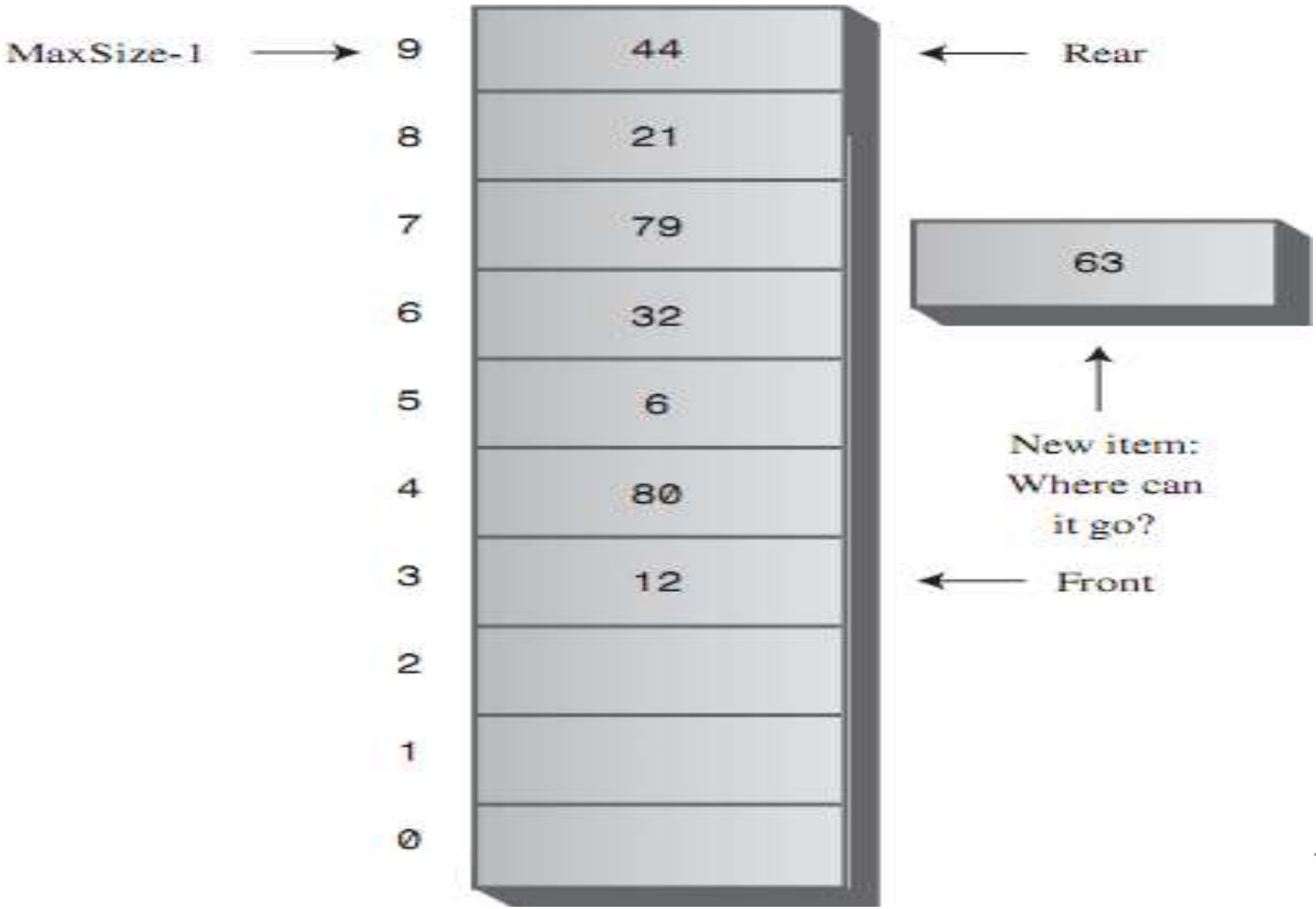

0 1 2 3 4 5 6
Array
Max = 6
Rear = -1
Front = -1
0 1 2 3 4 5 6
7
Rear = 0
Front = 0
After Insertion
0 1 2 3 4 5 6
7 8 9 10 11
Front = 0
Rear = 4
After Insertion
0 1 2 3 4 5 6
10
Front = 3
Rear = 3
After Deletion
0 1 2 3 4 5 6
10 11 12

Drawback of Linear Queue

The linear queue suffers from serious drawback that performing some operations, we can not insert items into queue, even if there is space in the queue. Suppose we have queue of 5 elements and we insert 5 items into queue, and then delete some items, then queue has space, but at that condition we can not insert items into queue.

The trouble with this arrangement is that pretty soon the rear of the queue is at the end of the array (the highest index). Even if there are empty cells at the beginning of the array, because we have removed them, we still can't insert a new item because Rear can't go any further. This situation is shown below.

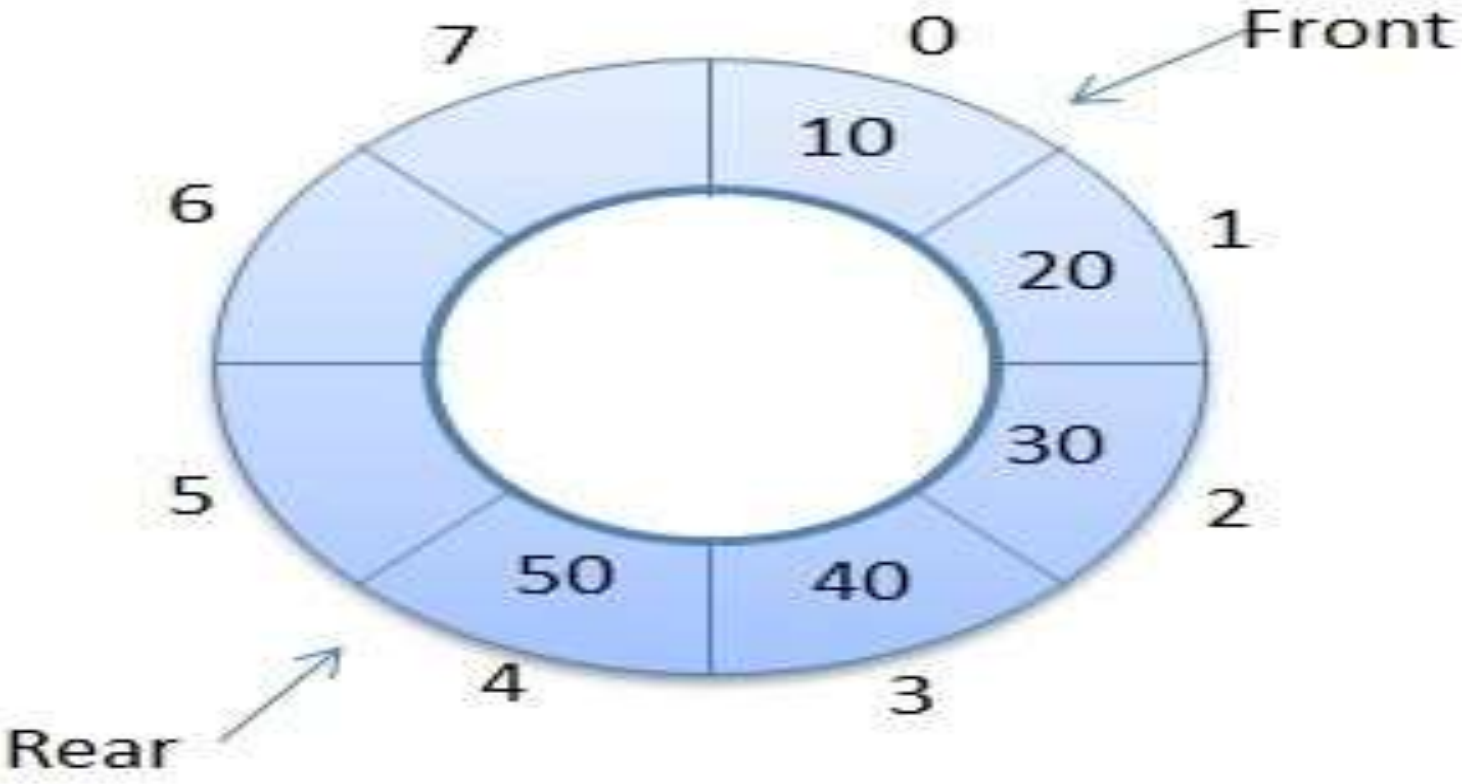

MaxSize-1
9
8
7
79
6
32
5
6
4
80
3
12
2
1
0
Empty cells
Rear
Front
Empty cells


MaxSize-1
9
8
7
6
5
4
3
2
1
0
44
21
79
32
6
80
12
Rear
63
New item:
Where can
it go?
Front

To avoid the problem of not being able to insert more items into the queue even when it's not full, the Front and Rear arrows wrap around to the beginning of the array. The results a circular queue(sometimes called a ring buffer).

What is Circular Queue?

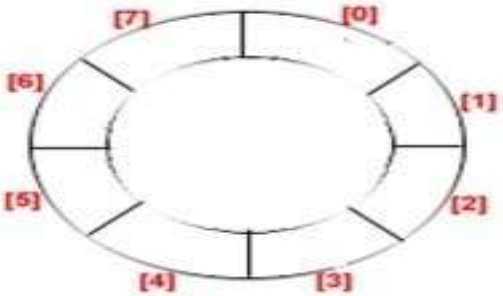
The queue is considered as a circular queue when the positions 0 and MAX-1 are adjacent. Any position before front is also after rear.Here Max=8



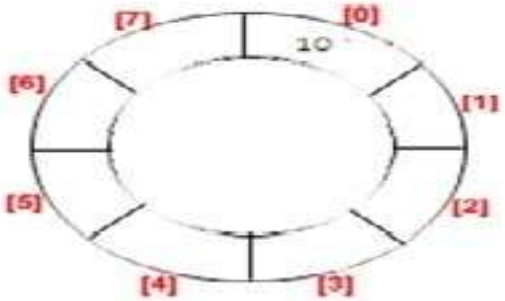
Note that the container of items is an array. Array is stored in main memory. Main memory is linear. So this circularity is only logical. There can not be physical circularity in main memory.

Consider the example with Circular Queue implementation:

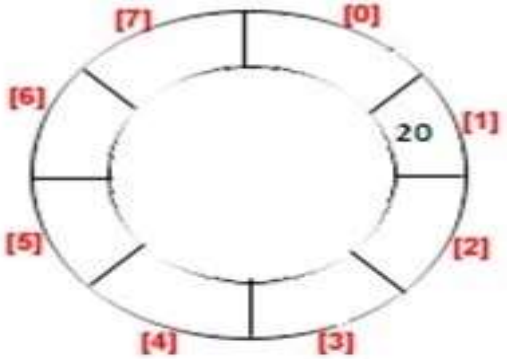
1) Initially: **Front = 0** and **rear = -1**



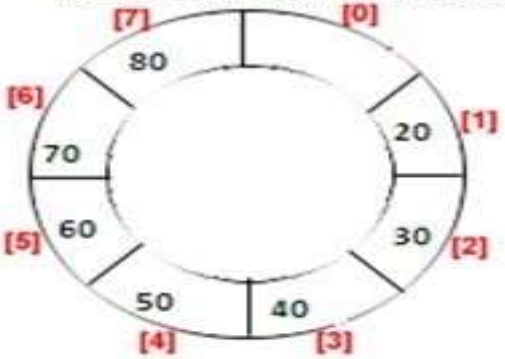
2) Add item 10 then **front = 0** and **rear =0**.



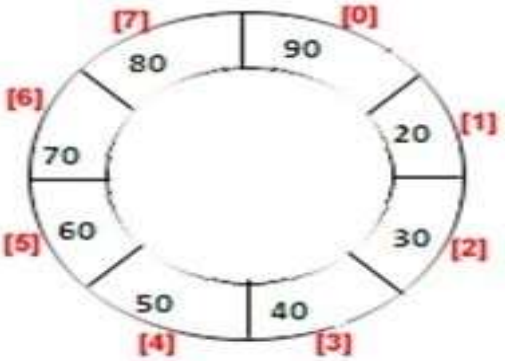
3) Now delete one item then **front = 1** and **rear =1**.



4) Like this now insert 30, 40, and 50,50,70,80 respectability then **front =1** and **rear = 7.**



5) Now in case of linear queue, we can not access 0 block for insertion but in circular queue next item will be inserted of 0 block then **front =0** and **rear = 0**.



See the logical circularity of the queue. Addition causes the increment in REAR. It means that when REAR reaches MAX-1 position then Increment in REAR causes REAR to reach at first position that is 0.

```
if( rear  == MAX -1 )
   rear  = 0;
else
   rear = rear + 1;
```

The short-hand equivalent representation may be

rear = (rear + 1) % MAX;

As we know that, Deletion causes the increment in FRONT. It means that when FRONT reaches the MAX-1 position, then increment in FRONT, causes FRONT to reach at first position that is 0.

```
if( front  == MAX -1 )
   front  = 0;
else
   front = front + 1;
```

The short-hand equivalent representation may be

front = (front + 1) % MAX;

Drawback of Circular Queue:

The drawback of circular queue is , difficult to distinguished the full and empty cases. It is also known as boundary case problem.

In circular queue it is necessary that:

Before insertion, fullness of Queue must be checked (for overflow).

Before deletion, emptiness of Queue must be checked (for underflow).

Condition to check FULL Circular Queue:

If **((rear == Max-1 && front == 0) || (front==Rear+1))**

Condition to check EMPTY Circular Queue:

if((front ==0 && Rear==-1)

Solution of the drawback of circular Queue:

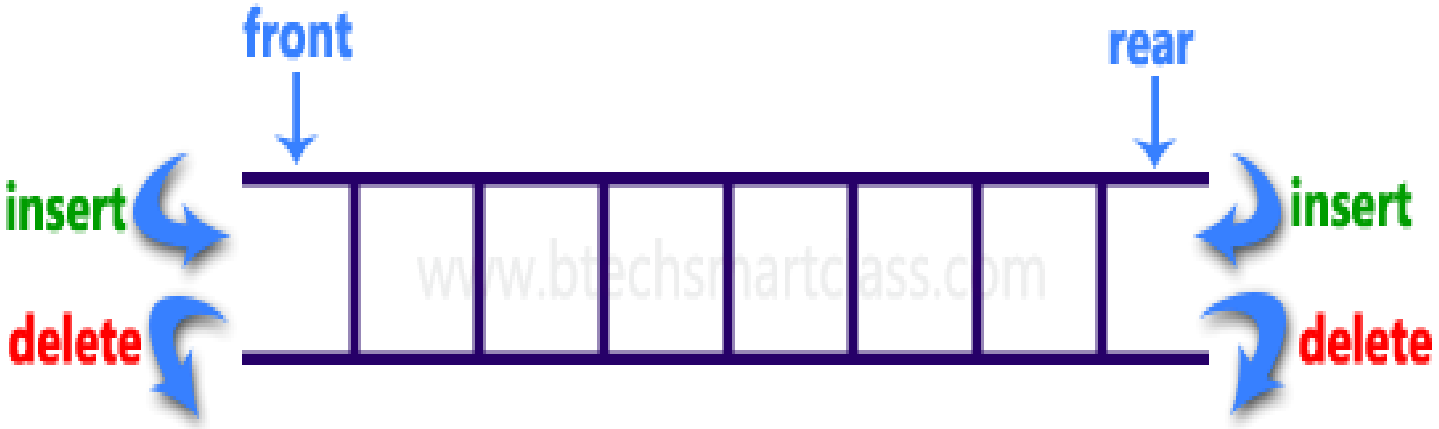
Use **count variable** to hold the current position (in case of insertion or deletion).

Operation of Circular Queue using count:

- Initialize Operation.
- Is_Full check.
- Is_Empty check.
- Addition or Insertion operation.
- Deletion operation.

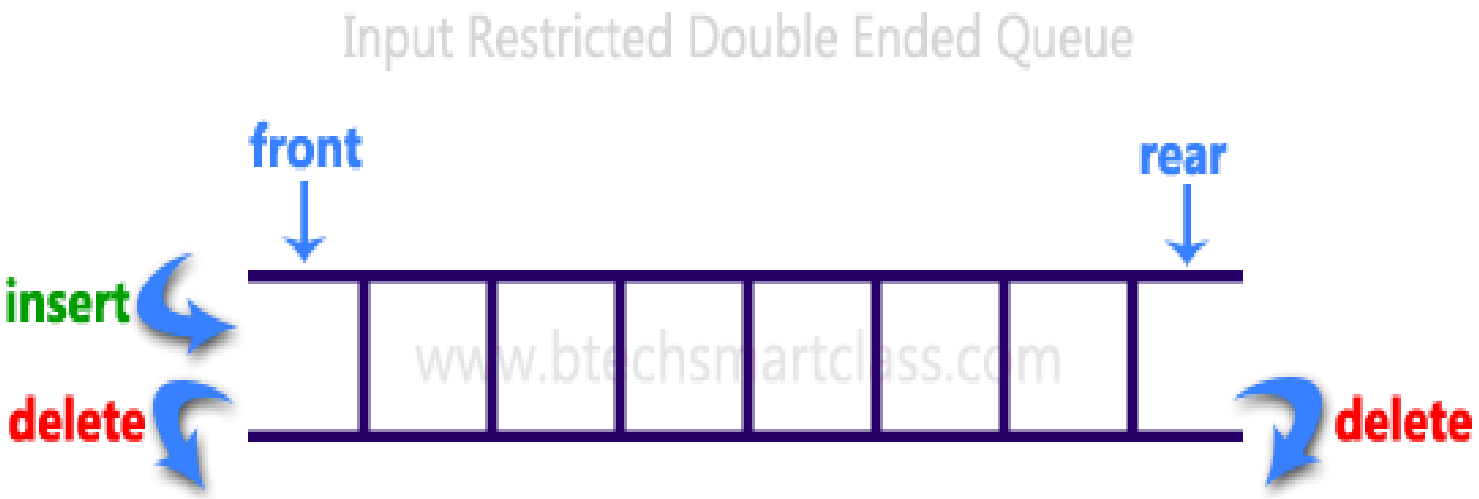
Double Ended Queue(Deque)

Double Ended Queue is also a Queue data structure in which the insertion and deletion operations are performed at both the ends (front and rear). That means, we can insert at both front and rear positions and can delete from both front and rear positions.



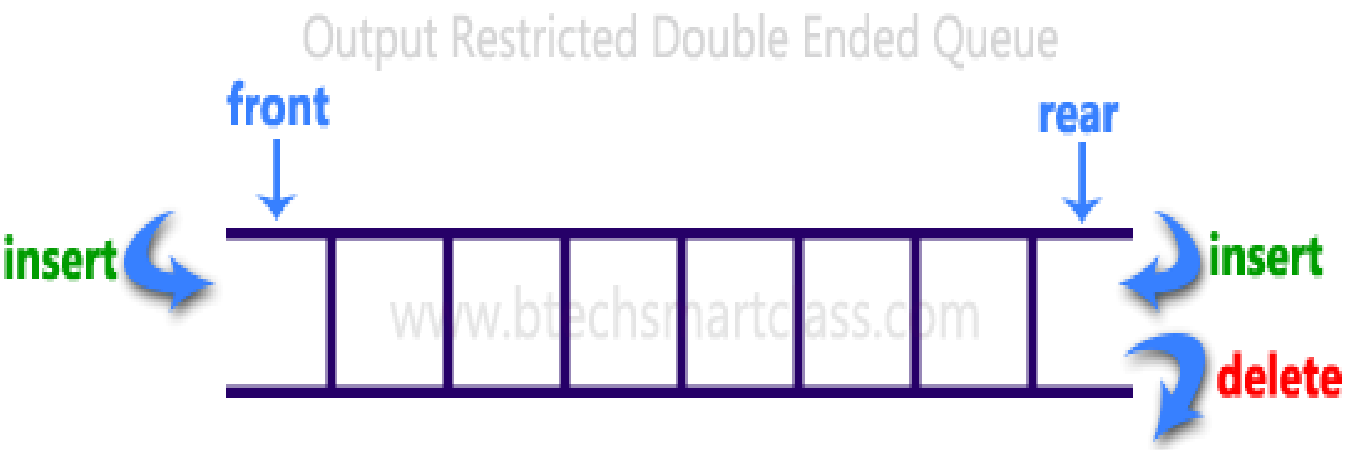
Input Restricted Double Ended Queue:

In input restricted double-ended queue, the insertion operation is performed at only one end and deletion operation is performed at both the ends.



Output Restricted Double Ended Queue:

In output restricted double ended queue, the deletion operation is performed at only one end and insertion operation is performed at both the ends.



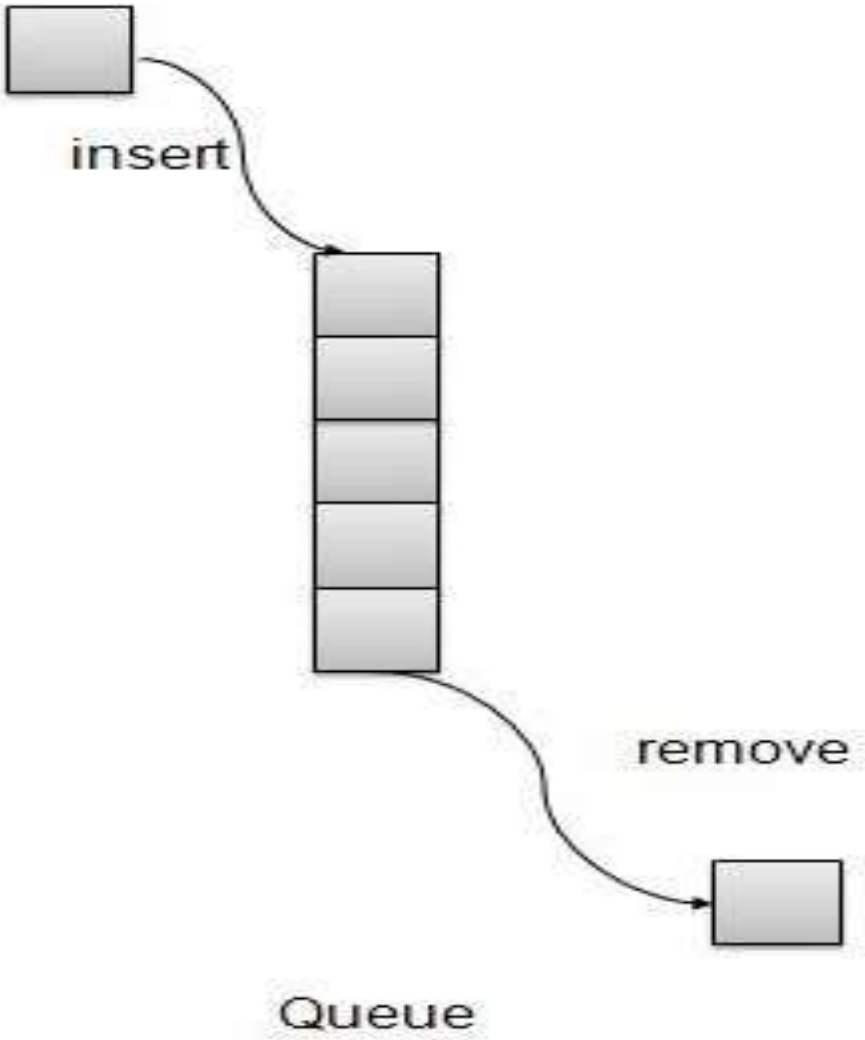
Priority Queue

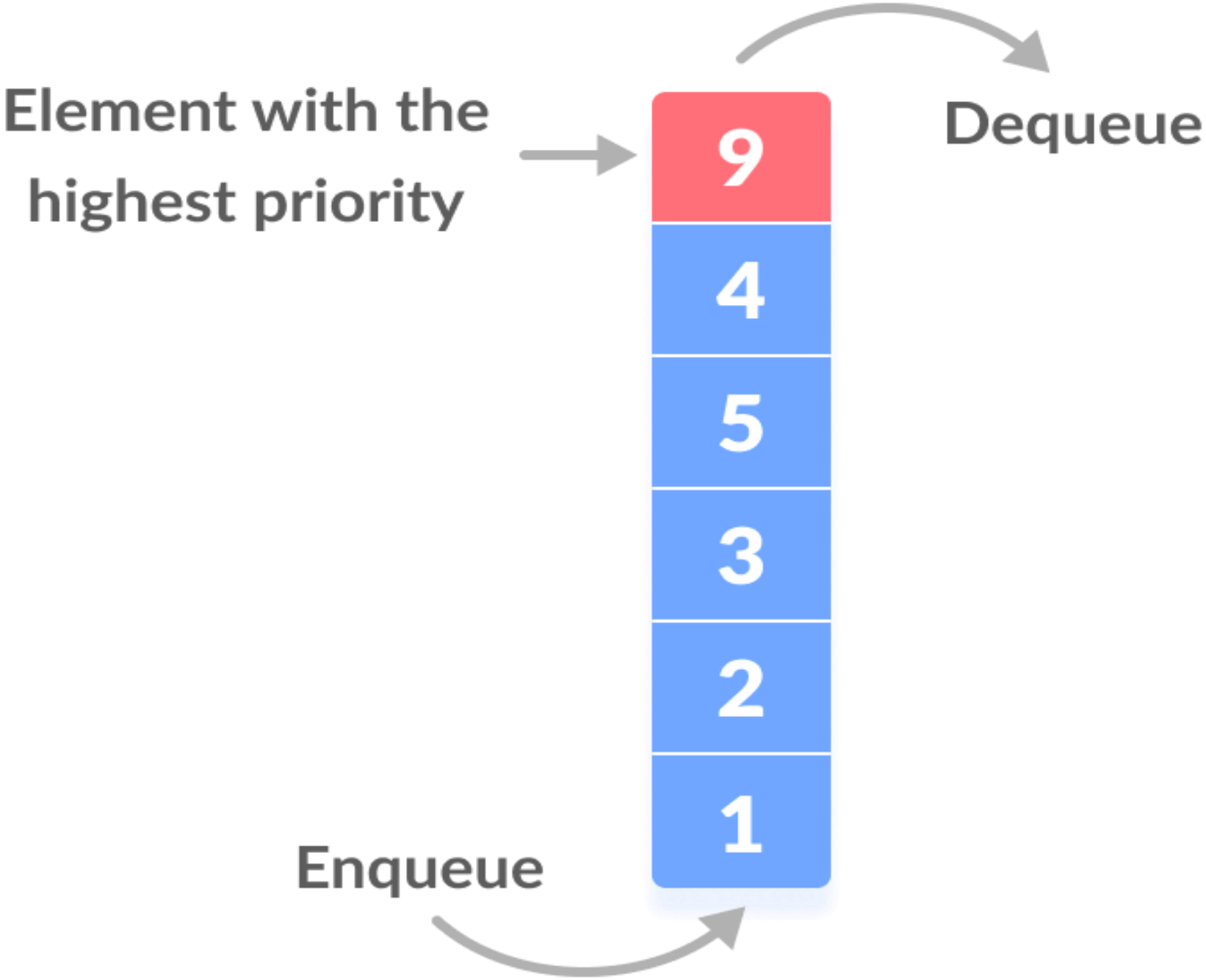
Priority Queue is more specialized data structure than Queue. Like ordinary queue, priority queue has same method but with a major difference. In Priority queue items are ordered by key value so that item with the lowest value of key is at front and item with the highest value of key is at rear or vice versa. So we're assigned priority to item based on its key value. Lower the value, higher the priority. Following are the principal methods of a Priority Queue.

Basic Operations

- insert / enqueue − add an item to the rear of the queue.
- remove / dequeue − remove an item from the front of the queue.

Priority Queue Representation:




Element with the highest priority
9
4
5
3
2
1
Dequeue
Enqueue

Applications of Queue

Queue, as the name suggests is used whenever we need to manage any group of objects in an order in which the first one coming in, also gets out first while the others wait for their turn, like in the following scenarios:

1. Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
2. In real life scenario, Call Center phone systems uses Queues to hold people calling them in an order, until a service representative is free.
3. Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive i.e First come first served.

Round Robin scheduling using Queue

Round Robin is a CPU scheduling algorithm where each process is assigned a fixed time slot in a cyclic way.

1. It is simple, easy to implement, and starvation-free as all processes get fair share of CPU.
2. One of the most commonly used technique in CPU scheduling as a core.
3. It is preemptive as processes are assigned CPU only for a fixed slice of time at most.
4. The disadvantage of it is more overhead of context switching.

Round Robin Example:

Process	Duration	Order	Arrival Time
P1	3	1	0
P2	4	2	0
P3	3	3	0

Suppose time quantum is 1 unit.

P1	P2	P3	P1	P2	P3	P1	P2	P3	P2

0 10

P1 waiting time : 4

P2 waiting time: 6

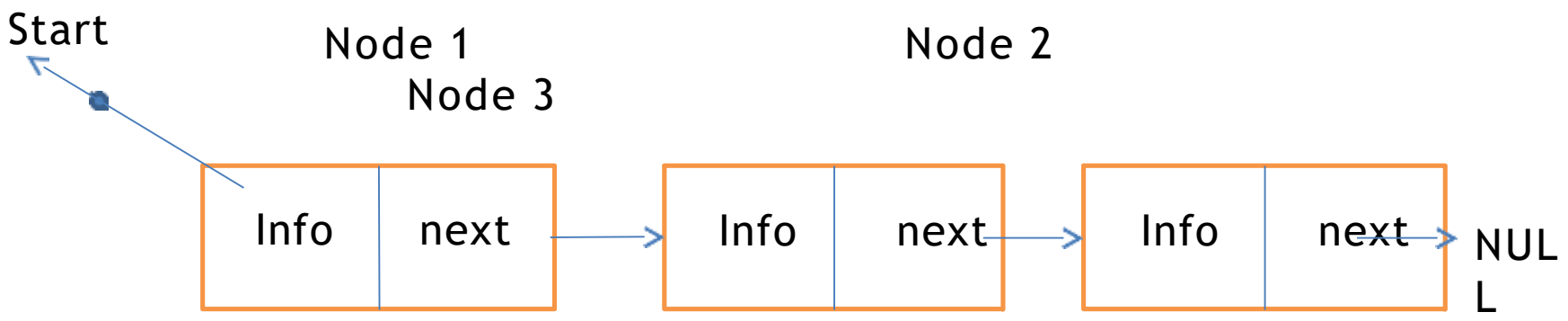
P3 waiting time: 6

The average waiting time(AWT) : (4+6+6)/3=5.33

Introduction to linked list

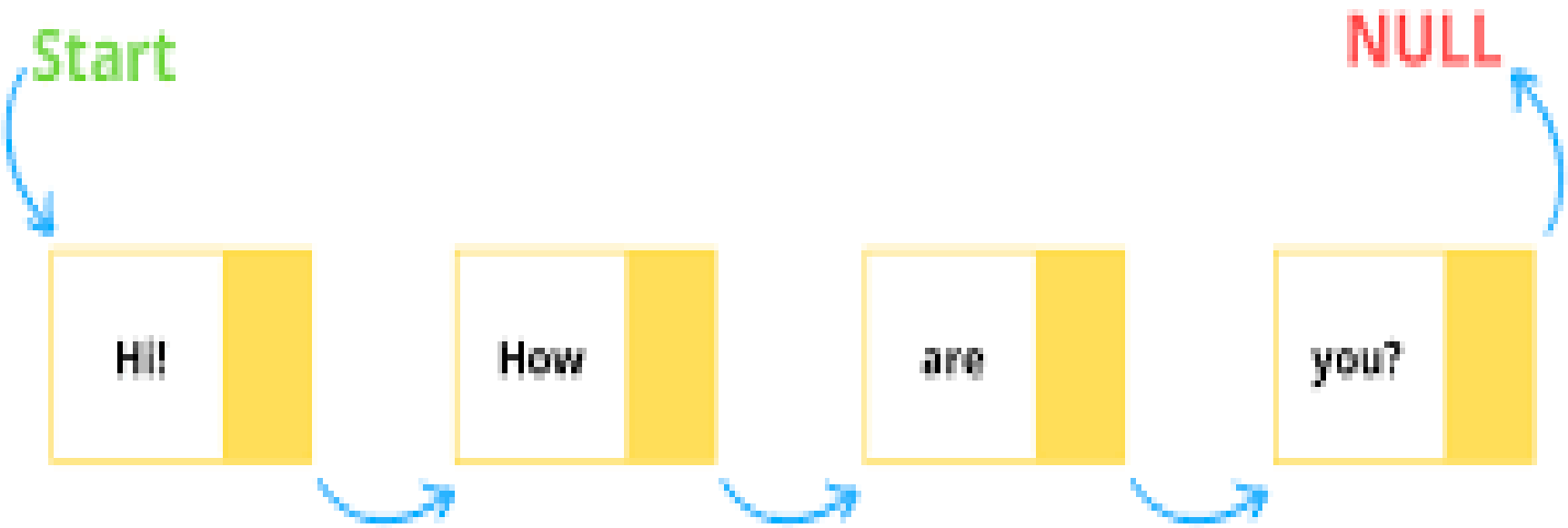
Definition of Singly linked list

- It is a special list of structures(node) which contain two part. **INFO** part stores the information and a **pointer** which points to the next element. It is dynamic in nature. Items may be added to it or deleted from it.
- **Example**




```
 if  Start= NULL
Singly linked list
List is empty
```

Linked list



Introduction to linked list cont..

- A list item has a pointer to the next element, or to Null if the current element is the tail (end of the list).

- This pointer points to a structure of the same type as itself.

- This structure that contains elements and pointers to the next structure is called a Node

Example of linked list:

- Hash tables use linked lists for collision resolution
- Any "File Requester" dialog uses a linked list
- Binary Trees
- Stacks and Queues can be implemented with a doubly linked list

Introduction to linked list cont..

Advantage and disadvantage of linked list:

Advantage

- Dynamic in nature, grow and shrink during execution time

- Efficient memory utilization that is, memory allocation and de-allocation as per requirement

- Insertion and deletions are easier and efficient that is, possible from any specified location

Disadvantage

- More memory space in needed to store large number of node than array
- Access to arbitrary node is time consuming and tedious

Introduction to linked list cont..

Operation on linked list:

Primary operation on linked list

- **Create:** One by one new node is created to create linked list
- **Insertion:** Insert a new node at specified location in the list
- **Deletion:** A specified node from the list
- **Traversing:** Visiting the entire list node and finding specified node in the list
- **Concatenation:** Joining one list at the end of the other list

Introduction to linked list cont..

Types of linked list

Following are the categories of linked list

- Singly- linked list
- Doubly- linked list
- Circular linked list
- Circular doubly linked list

Singly linked list

Representation of singly linked list

Singly linked list cab be represented in memory with following declaration

This declaration define new data type

```
struct node
{ int i;
struct node *next;
}
typedef struct node NODE;
NODE *start;
```

Singly linked list cont..

Creating a node of linked list

- Using Pointers , structures and dynamic memory allocation **malloc** function
- C provides following functions for dynamic memory allocation and de-allocation

1. malloc():

- This function allocates memory block in bytes and returns void pointer to first block
- So type casting is required to convert void pointer in the required type
- E.g. int *ptr;

```
ptr= (int *) malloc(10 *sizeof(int));
```

Singly linked list cont..

- **Memory allocation for structure variable**

```
 struct node
{ int item;
struct node  * next;
};
struct node  * ptr_var;
ptr_var = (struct node *) malloc(sizeof(struct node)
```

- If we write syntax

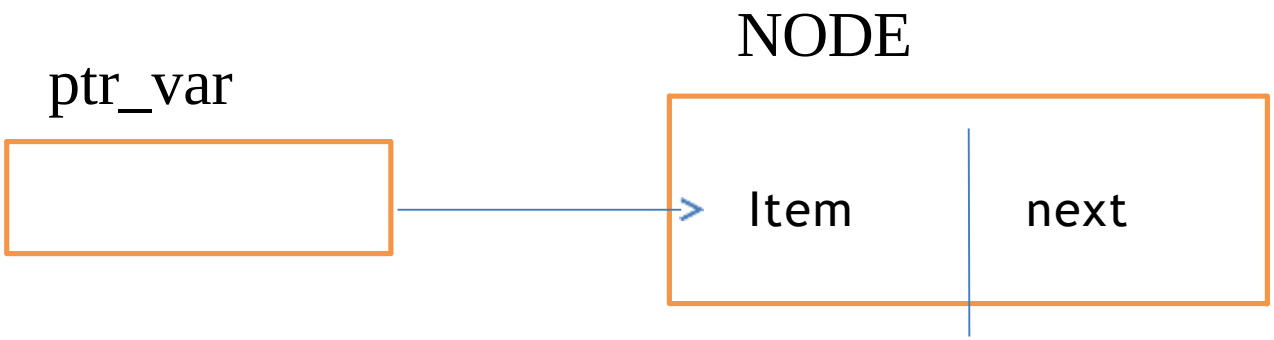
typedef struct node NODE;

Then

```
 ptr_var =(NODE *) malloc(sizeof(NODE));
```

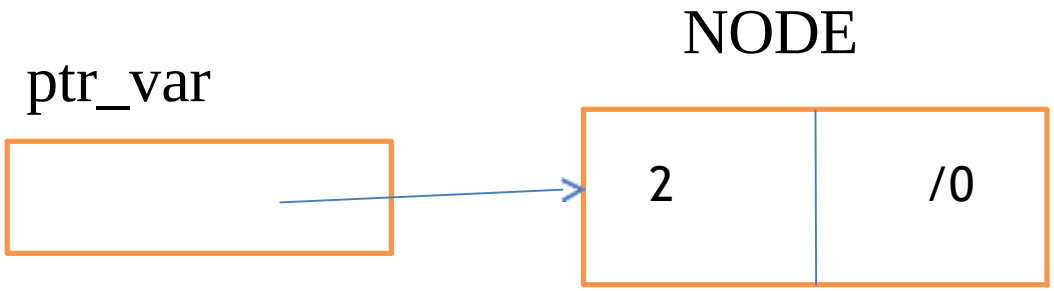
Singly Linked list cont..

- **Memory allocation for structure variable**



Assigning values to the field of NODE

- Ptr_var-->item=2;
- Ptr_var-- >next=‘\0’;



Singly Linked list operations

Inserting the node in the linked list

1. Inserting at the beginning of the list
2. Inserting at the end of the list
3. Insertion at the specified position within the list

Algorithm Notations

- **node(p):** a node pointed to by the pointer **p**
- **item(p):** item of the node pointed by pointer **p**
- **next(p):** next field of the **node(p)**

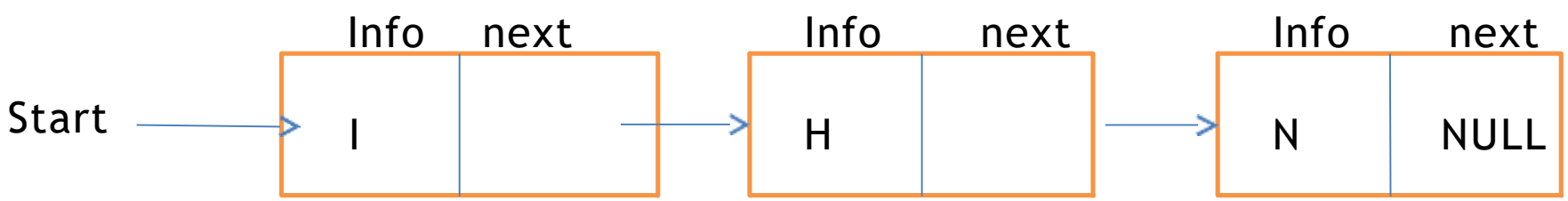
Algorithm steps for inserting a node to the List

Common steps are

- allocate space for a new node using **malloc()**
- copy the item into it,
- make the pointers adjustments

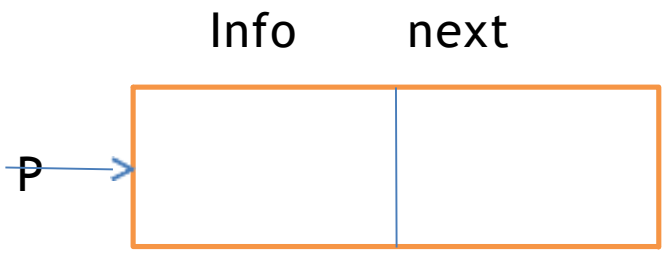
Inserting element at the beginning of the list

Assume the following list



Singly Linked list operations cont..

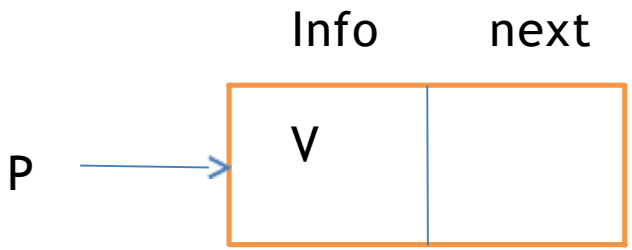
- allocate space for a new node using **malloc()**

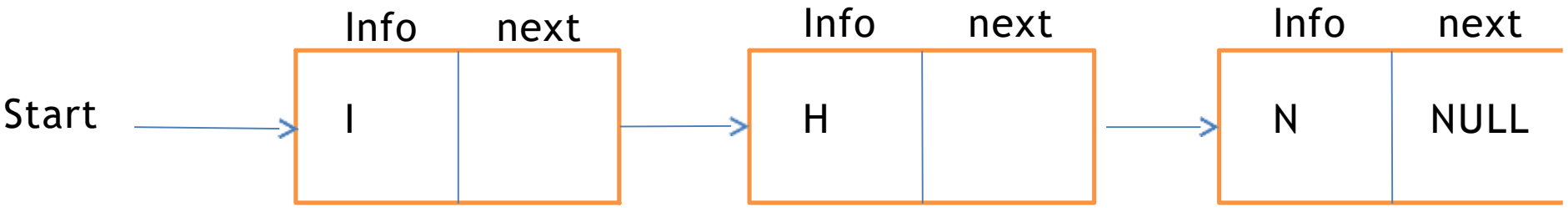


- copy the item into it,

p- ->info = V;

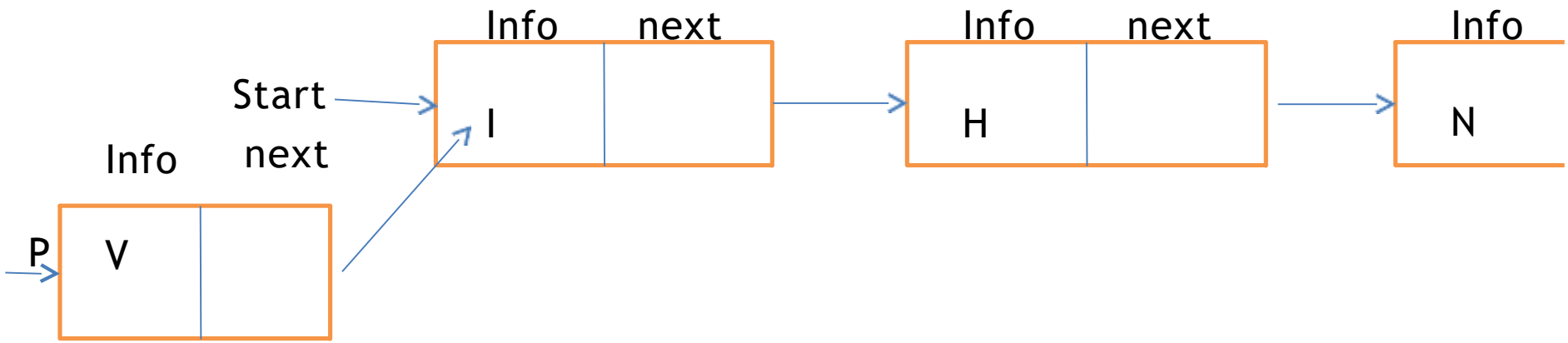
- make the pointers adjustments



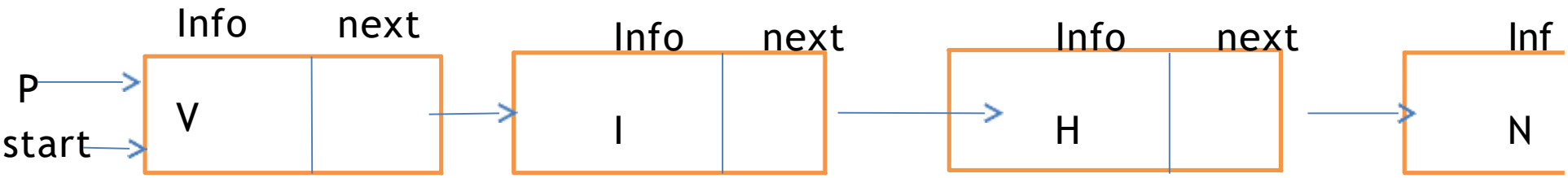


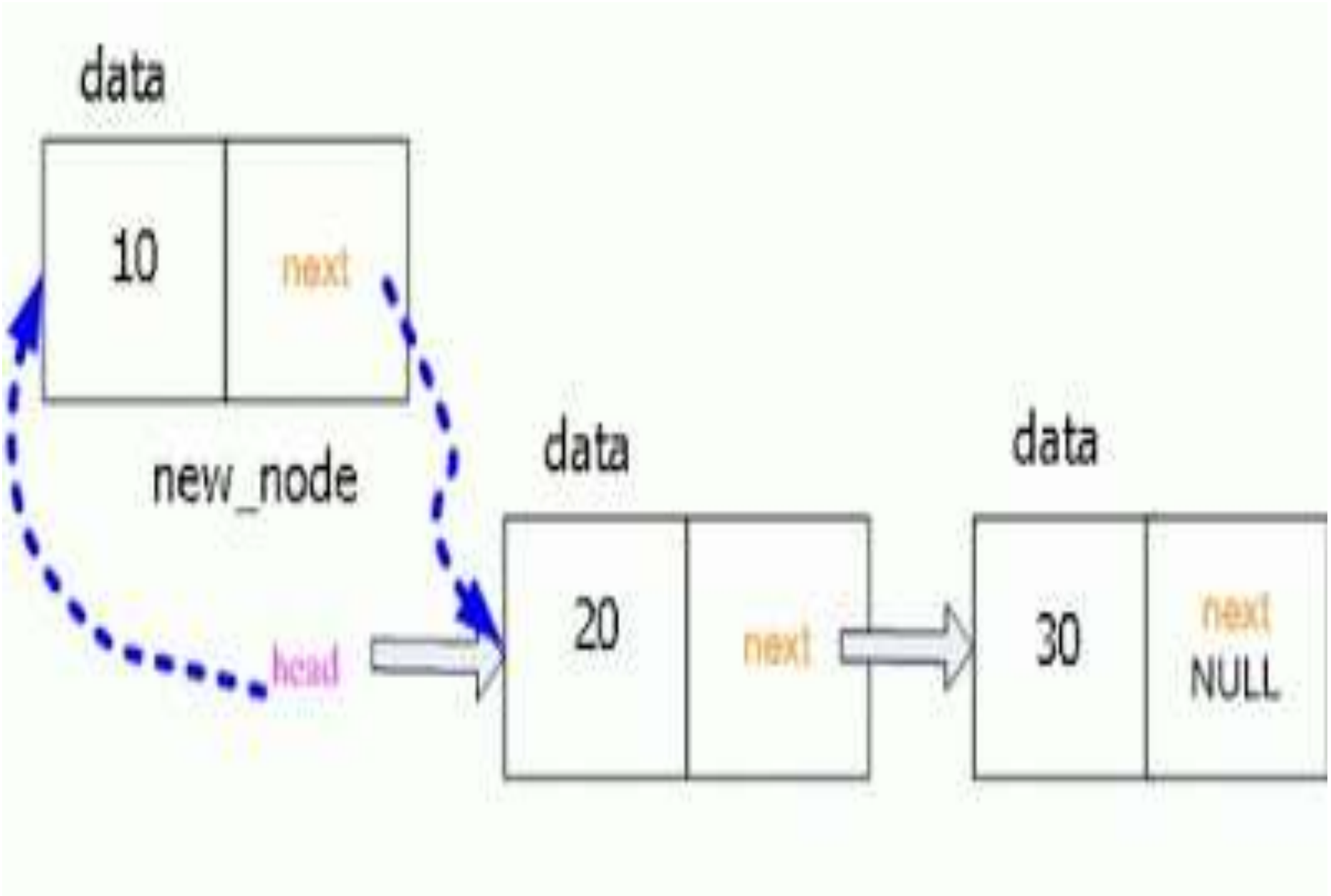
Singly Linked list operations cont..

- next(p) = start;



- start = p;

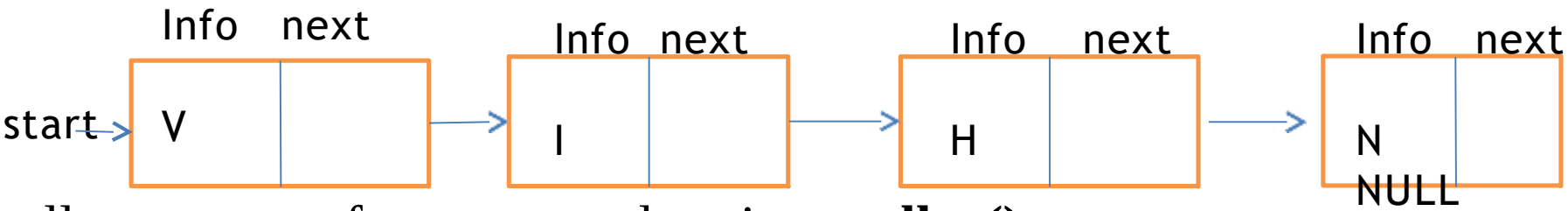



data
10
next
new_node
head
data
20
next
data
30
next
NULL

Singly Linked list operations cont..

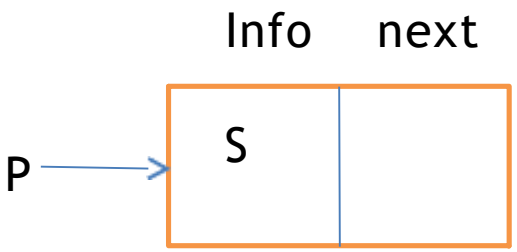
Inserting at the specified position within the list

- **Assume the following list**



- allocate space for a new node using **malloc()**
- copy the item into it,

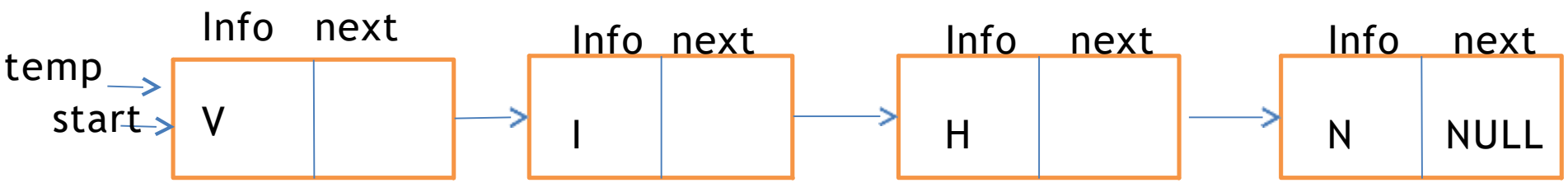
p- ->info = S;



Singly Linked list operations cont..

- **make the pointers adjustments**
- Assume we want to insert a node between Node I and Node H
- So to find that location

Set counter c=0 and pointer Node * temp
Set temp=start

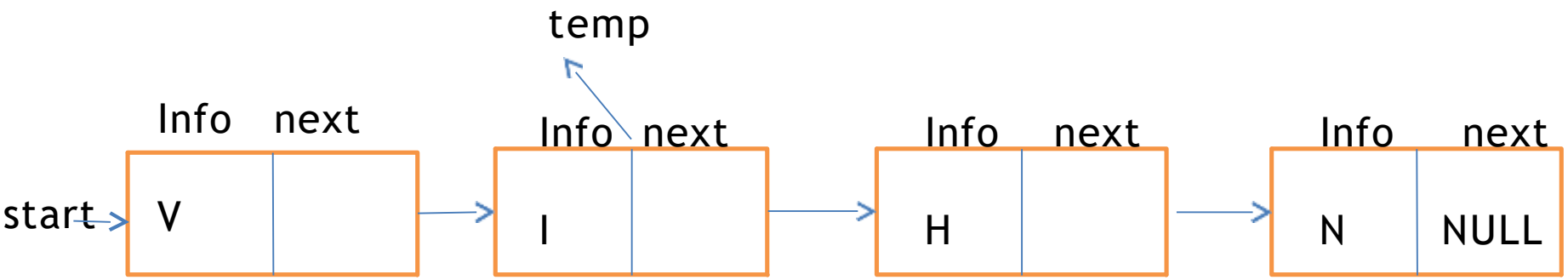


Singly Linked list operations cont..

Now to find the location, repeatedly do

Set temp =temp->next till c<location

Set c=c+1

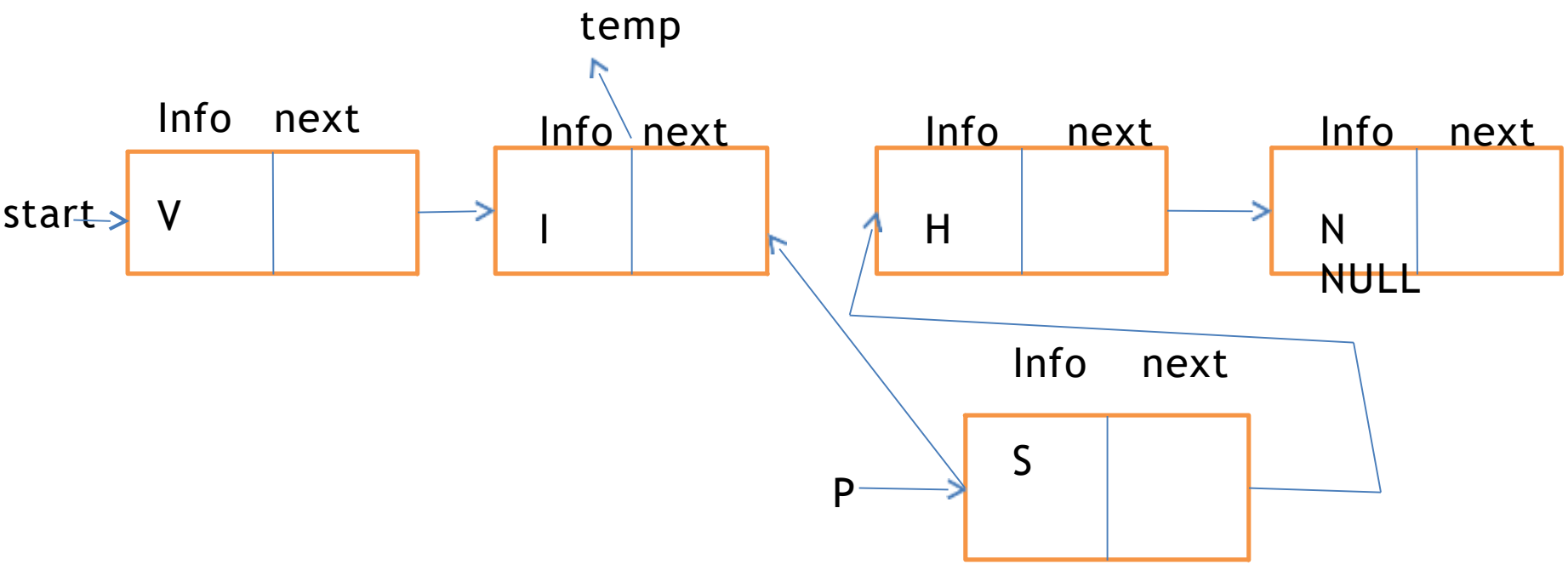


Singly Linked list operations cont..

then

Set p-> next = temp->next

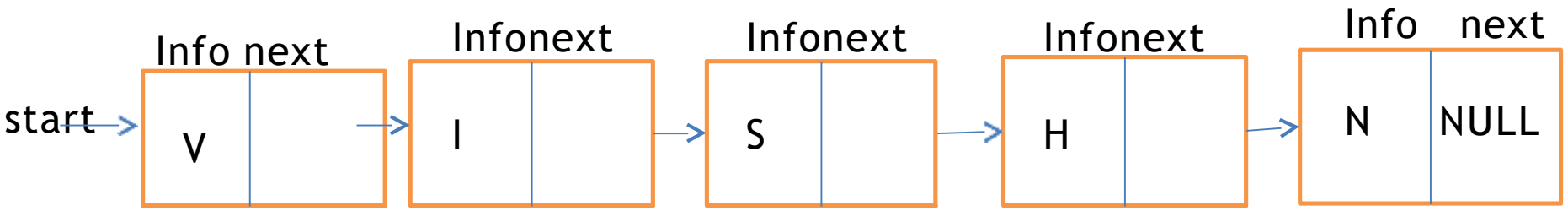
Set temp->next = p



Singly Linked list operations cont..

Inserting at the end of the list

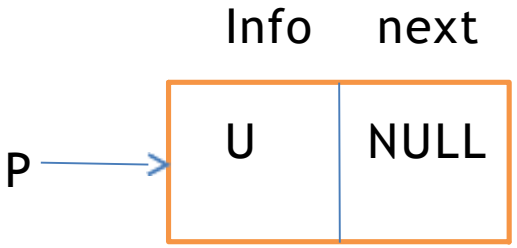
- Again assume the following list
- allocate space for a new node using **malloc()**



- copy the item into it,

p- ->info = U;

P- ->next=NULL;



Singly Linked list operations cont..

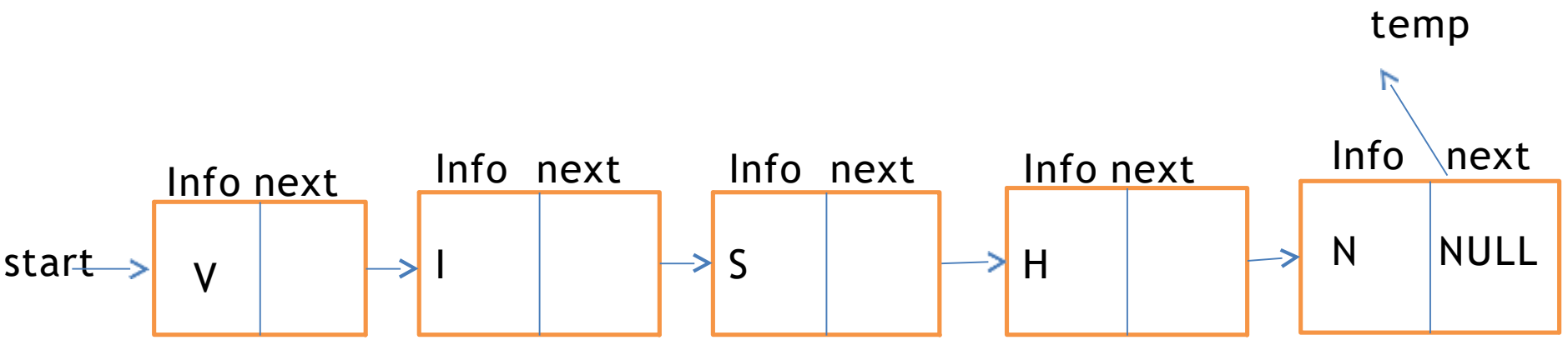
- So to find that location

Set pointer Node * temp

Set temp=start

Now to find the location, repeatedly do

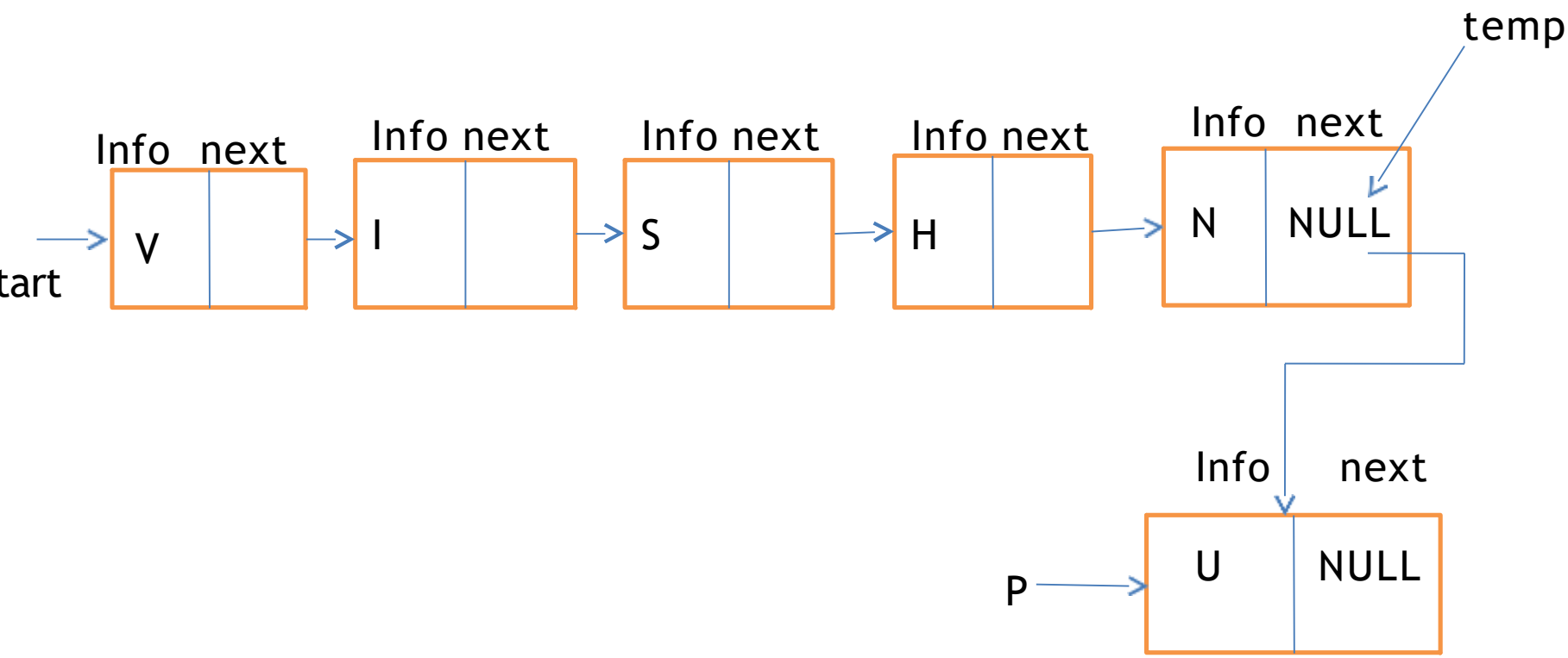
Set temp =temp->next untill temp- ->next!=NULL



Singly Linked list operations cont..

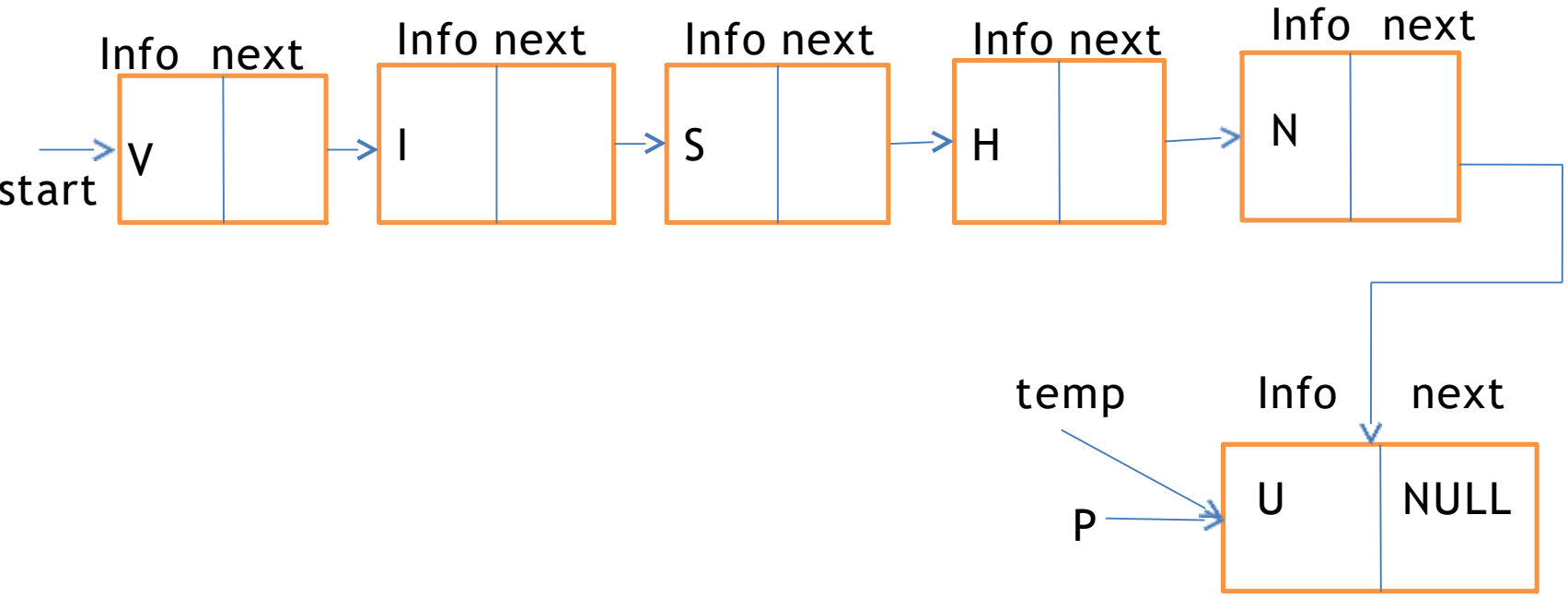
then

Set temp->next = p



Singly Linked list operations cont..

- **Set** temp = temp->next



Singly Linked list operations cont..

Linked list node deletion

1. Deleting the start node of the list
2. Deleting the end of the list
3. Deletion at the specified position within the list

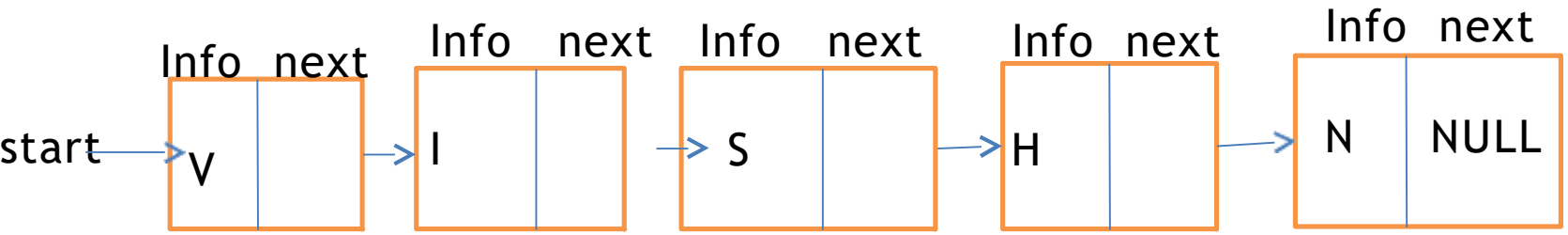
Algorithm steps for deleting a node to the List
Common steps are

- Search the node by setting and moving the pointer
- Take the information of info field of desired node in temporary variable
- Make pointers adjustments

Singly Linked list operations cont..

Deleting first element of the list

Assume the following list



If start =NULL then

Empty list

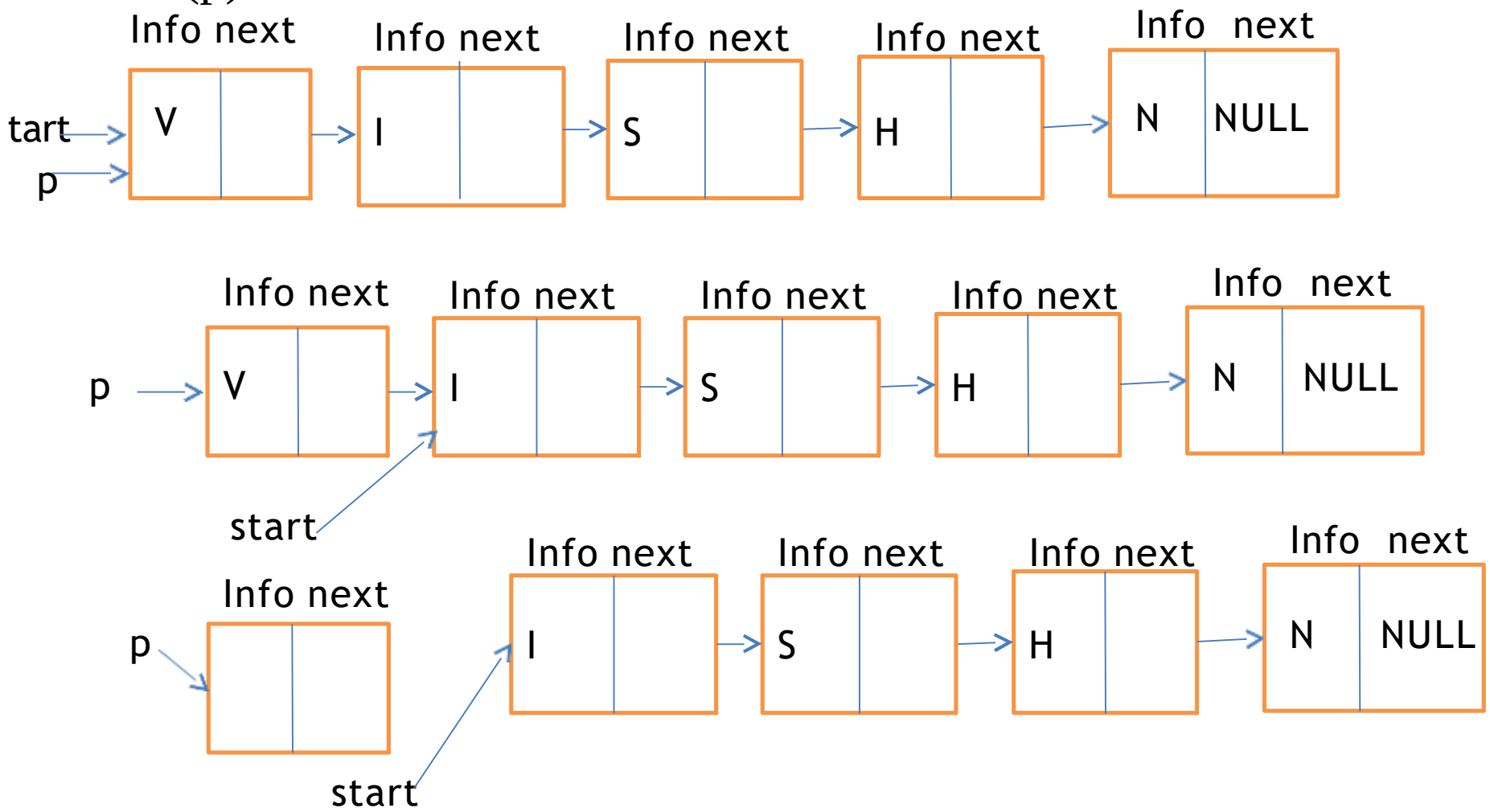
Else

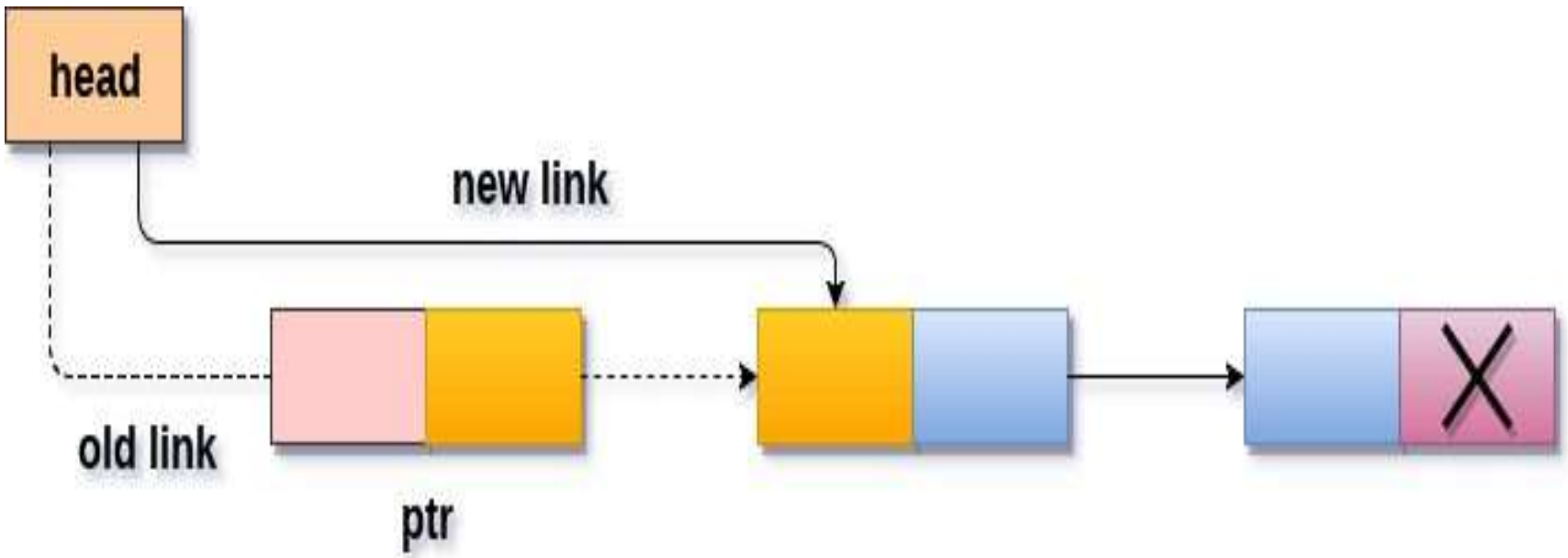
Set p = start

Set start = start- -> next

Singly Linked list operations cont..

- Char var = p- ->info
- p->Next=NULL
- free(p)





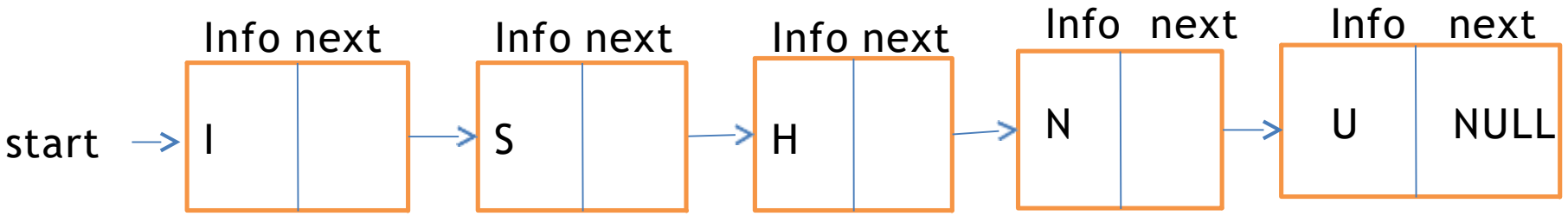

```
ptr = head
head = ptr -> next
free(ptr)
```

Deleting a node from the beginning

Singly Linked list operations cont..

Deleting the last element of the list

Assume the following list



If start = NULL **then**

Empty list

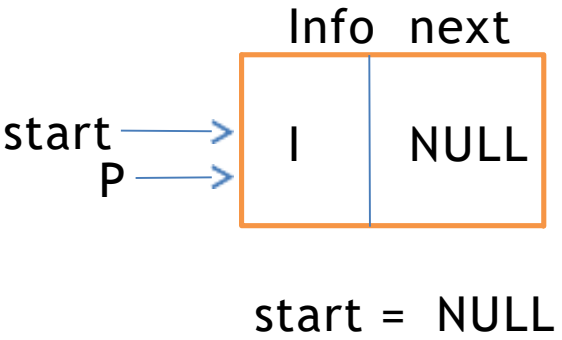
If start - -> next=NULL then

Set p = start

Set start = NULL

Char var = P - -> info

Free(p)



Singly Linked list operations cont..

else

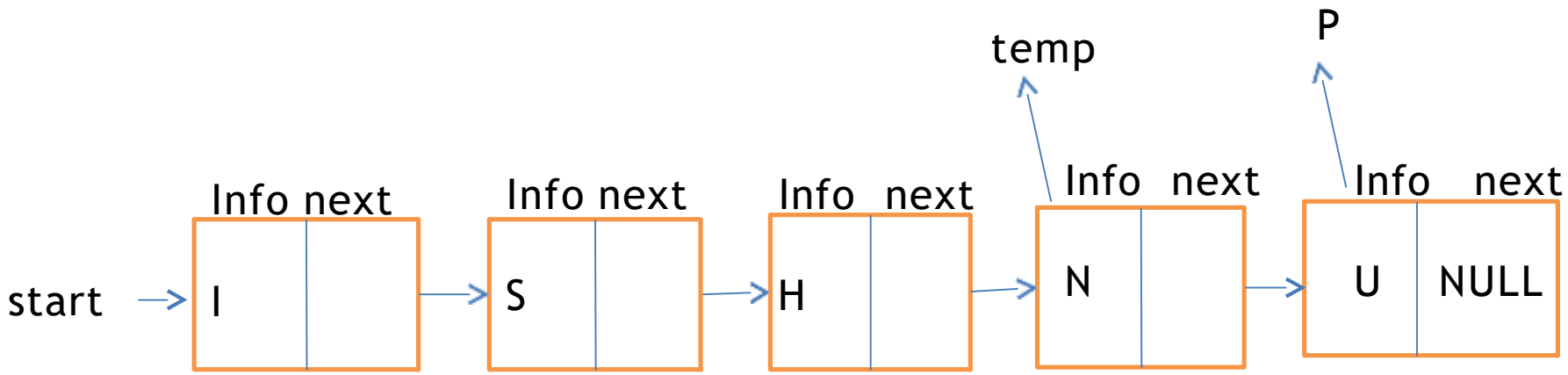
set p = start

Now to find the location, repeatedly do

until p- ->next ! = NULL

Set temp = p

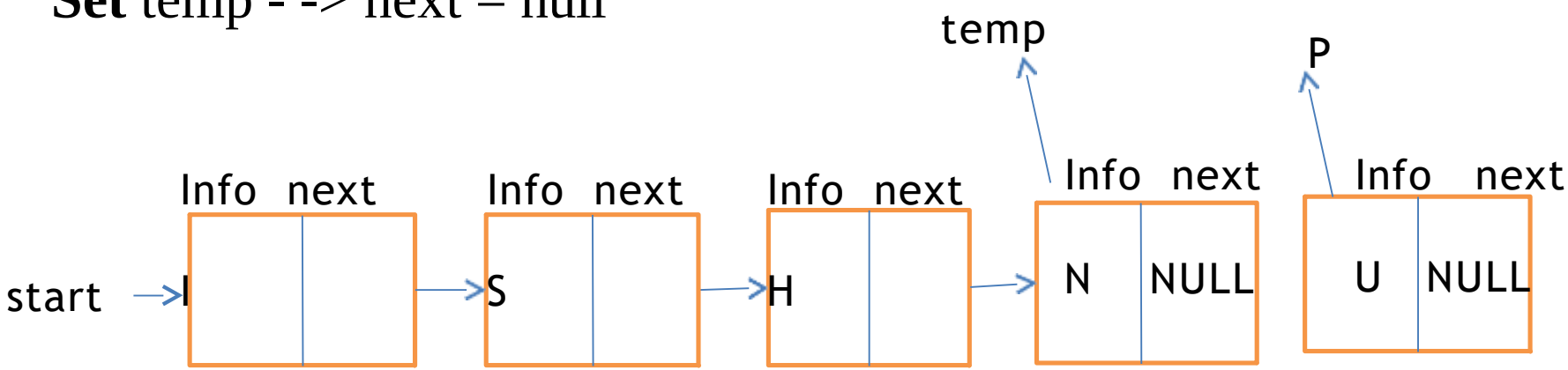
Set p = p - ->next



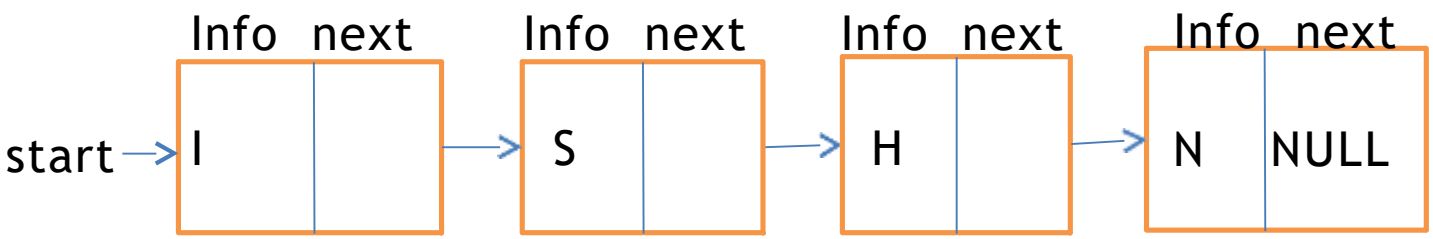
Singly Linked list operations cont..

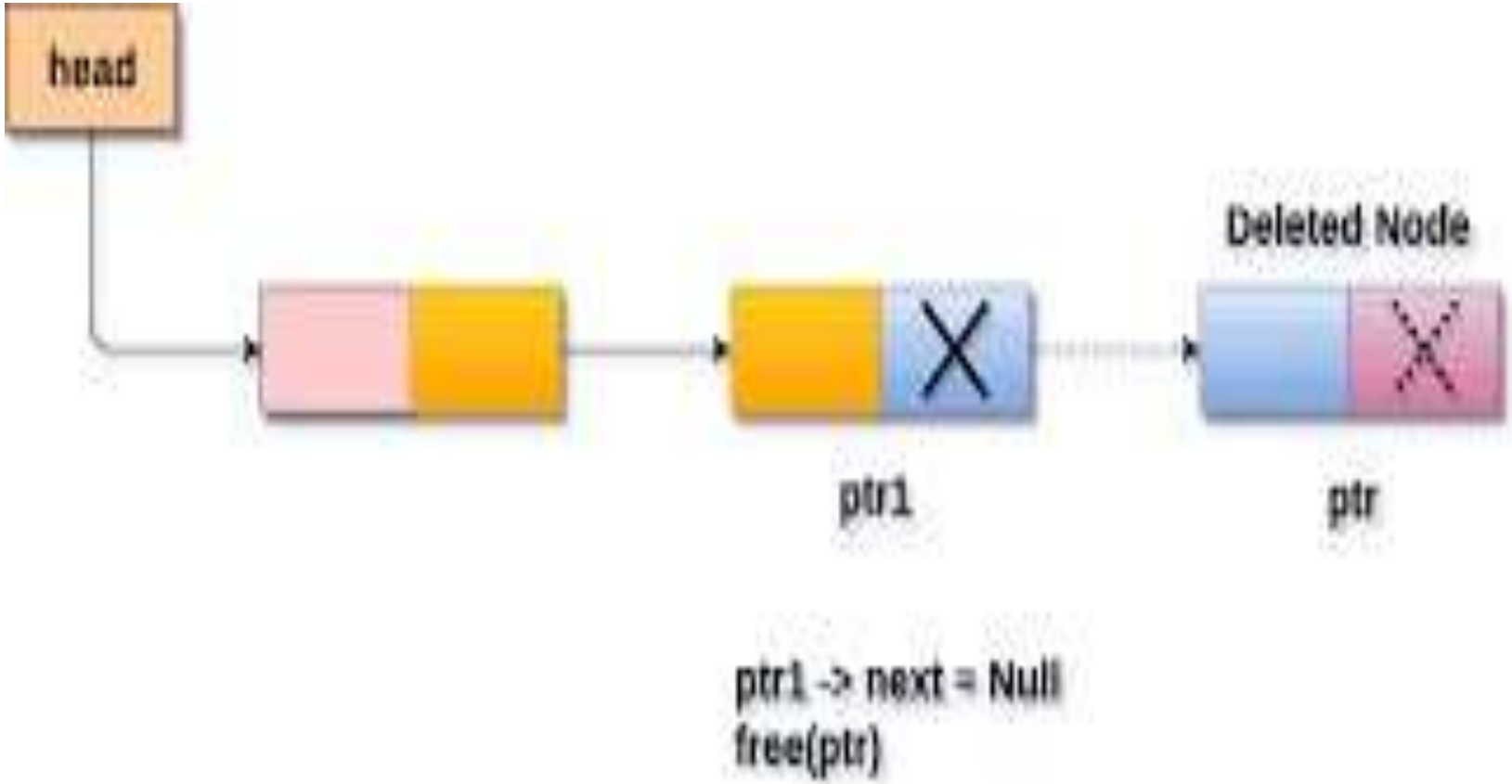
then

Set temp - -> next = null



Free(p)



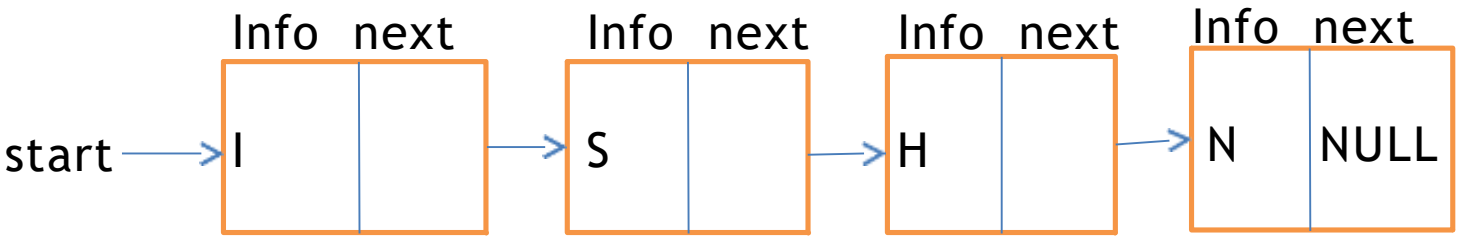


Deleting a node from the last

Singly Linked list operations cont..

Deletion at the specified position within the list

Assume the following list



Suppose we want to remove node at **3rd** position from start that is node H

If start = NULL **then**

Empty list

So to find that location

Set counter and pointer

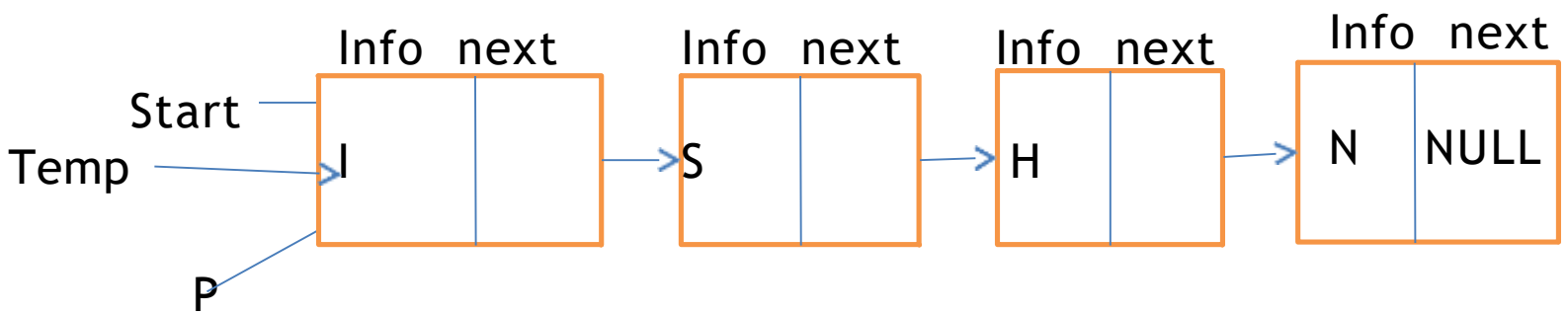
Set c=1

Set Node * temp , p

Set p=start

Singly Linked list operations cont..

Repeat following steps until c < location
Set temp = p
Set p = p --> next
Set c=c+1
Print p - -> info

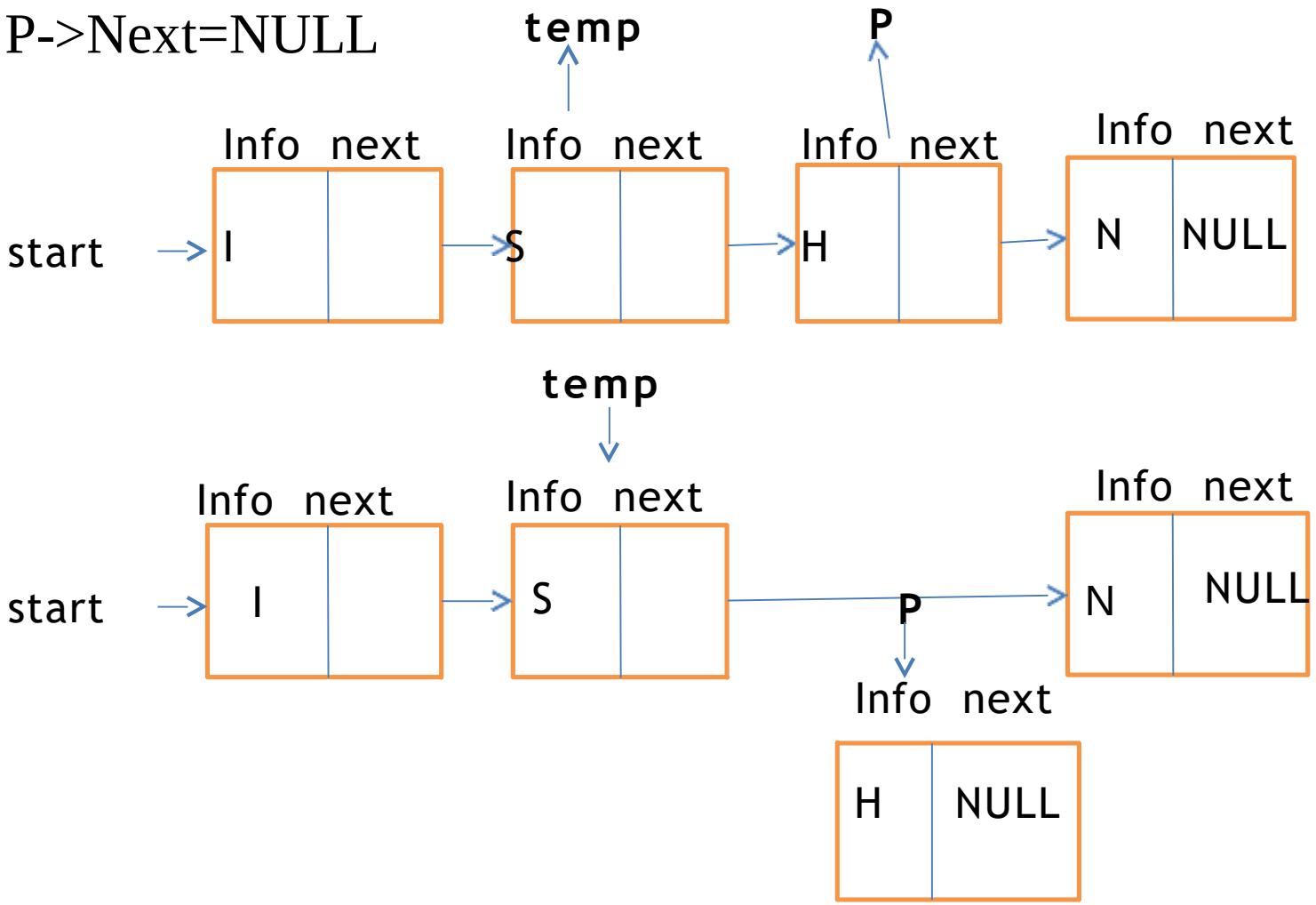


Singly Linked list operations cont..

At this stage C==3 required position

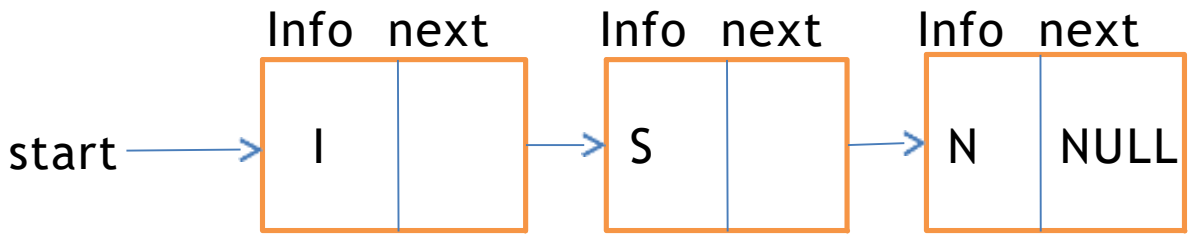
Set temp - ->next = p - -> next

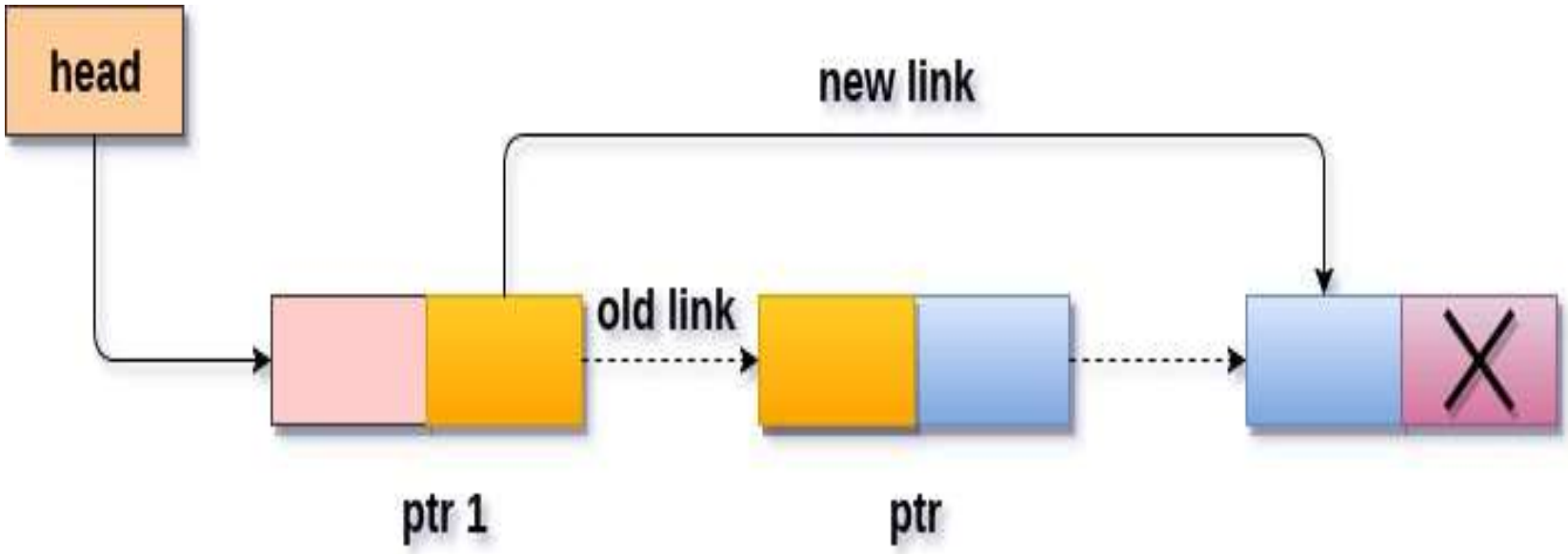
Set P->Next=NULL



Singly Linked list operations cont..

After Free(p)
Finally we have






```
ptr1 -> next = ptr -> next
free(ptr)
```

Deletion a node from specified position

Advantage and disadvantage of single linked list

Advantage

- Forward direction accessing of nodes are easy

- Insertion and deletions are easier and efficient that is, possible from any specified location

Disadvantage

- Forward traversal does not allow accessing the preceding node of the current node

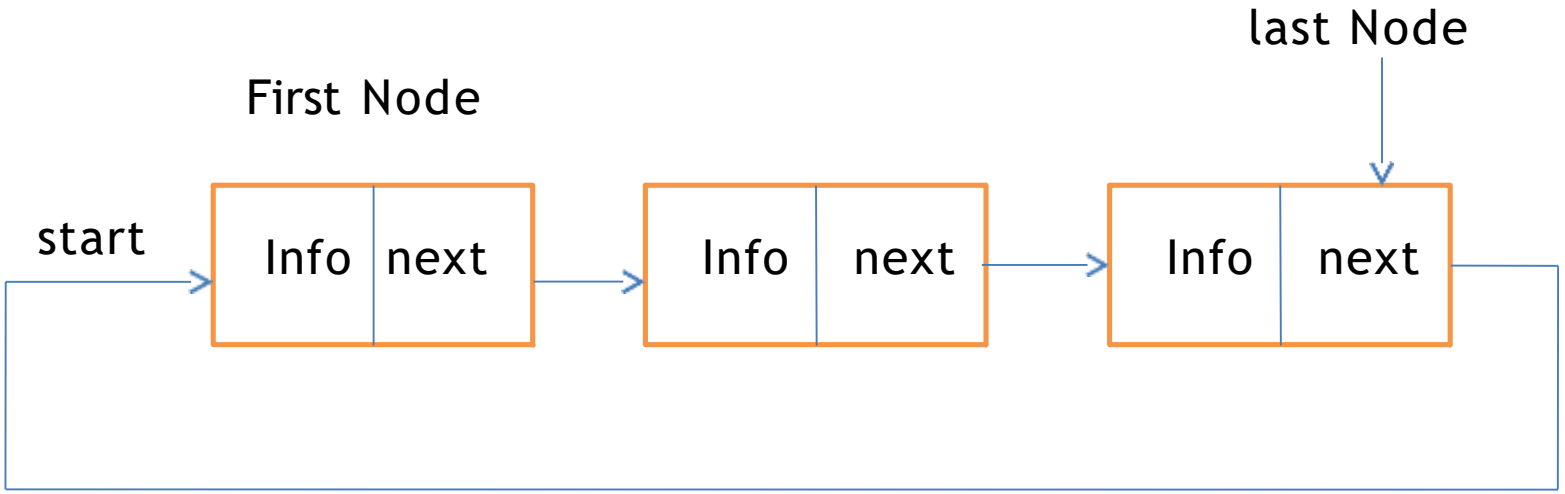
- Access to arbitrary node is time consuming and tedious

Circular linked List

Definition

- Circular lists are like singly linked lists, except that the last node contains a pointer back to the first node rather than the null pointer.

Example



Circular linked List cont..

- Possible to reach from any point in such a list, to any other point

- Does not have a natural "first or "last" node

- Let, external pointer **start** point to the First node, and the following node be the Last node

Representation of circular linked list

```
struct node
{ int info;
struct node *next;
};
typedef struct node NODE;
NODE  *start= NULL;
NODE  *last= NULL
```

Circular linked List operations

Insertion and deletion in circular linked list

Inserting node at beginning

allocate space for a new node using **malloc()**

- copy the item into it,

p- ->info = V;

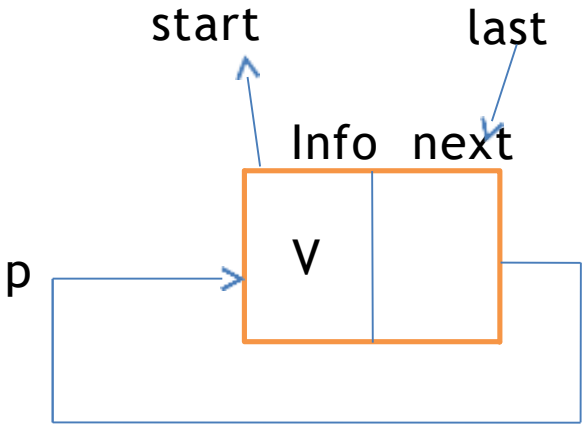
- make the following pointers adjustments

If start= = NULL

P - -> next = p;

start = p;

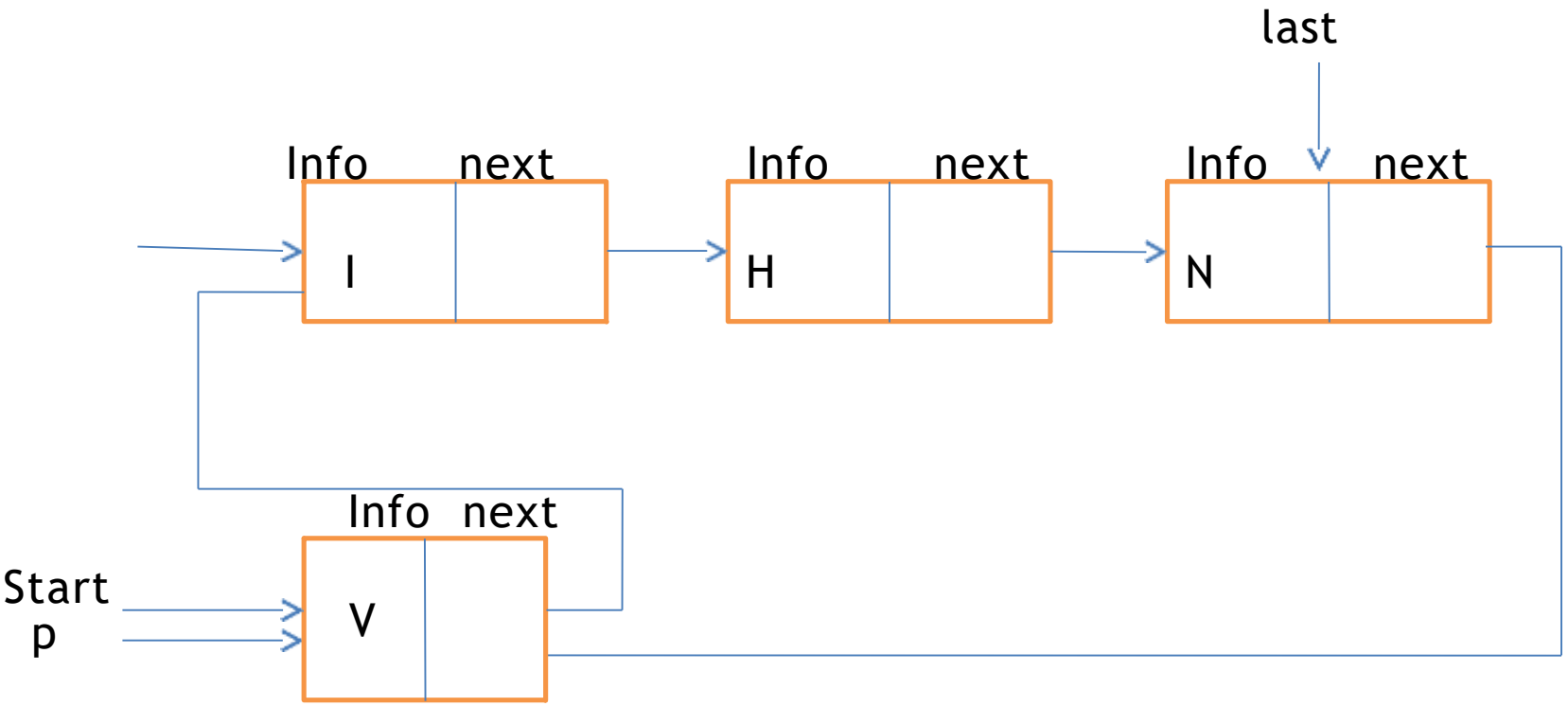
last = p;



Insertion and deletion in circular linked list cont..

else

- p - -> next = start;
- start = p;
- last - - > next = p;

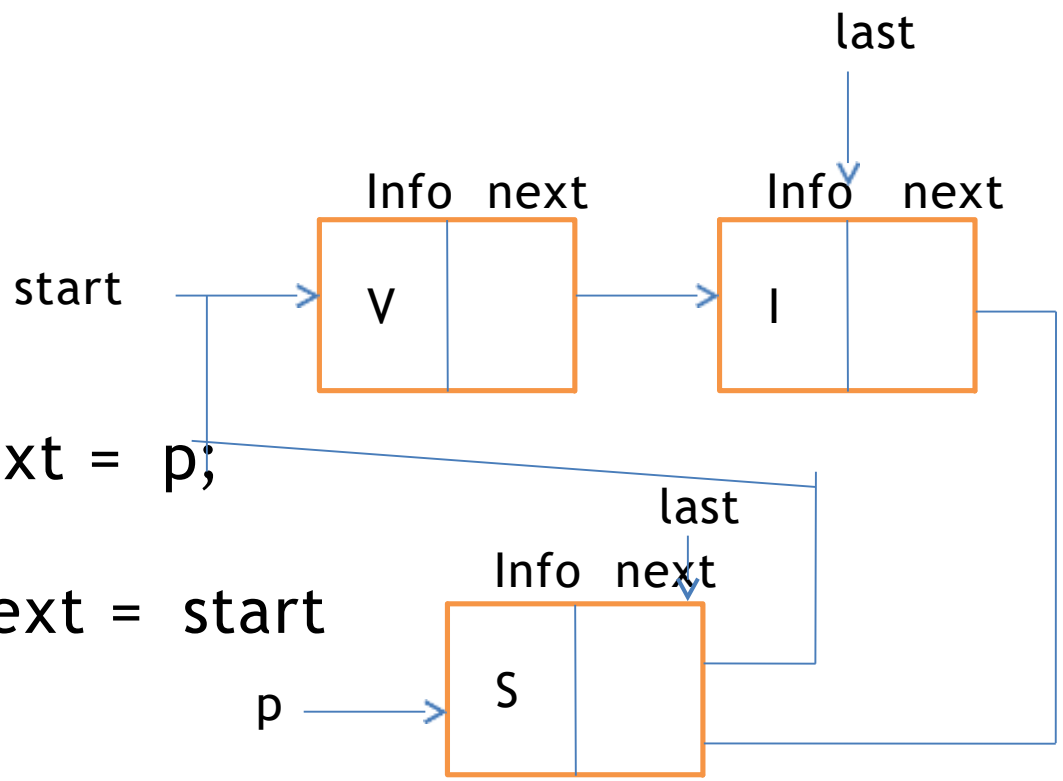


Insertion and deletion in circular linked list cont..

Inserting node at the end of list

If start = NULL **then**
Set P - -> next = p;
set last = p;
set start = p;

else




```
Set last - -> next = p;
Set last = p;
Set last - - > next = start
```

Insertion and deletion in circular linked list cont..

Algorithm for deleting a node from the beginning of the list

If start = NULL

list empty

else

Set p = start

Set start = start - - > next

Char var = p - -> info

Set last - -> next = start

Free(p)

Note: In same way other operations can also be performed on circular list

Advantage and disadvantage of circular linked list

Advantage

- Possible to reach from any point in such a list, to any other point
- Able to add or remove an element from either the front or rear of a list

Disadvantage

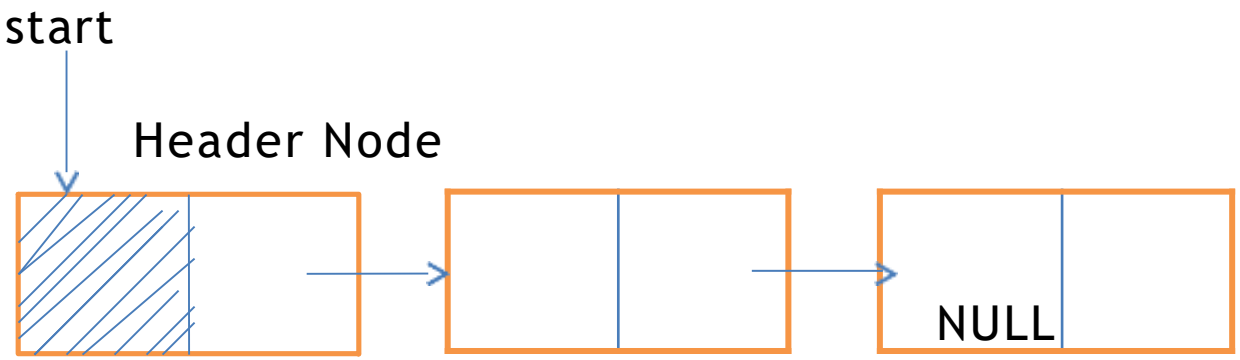
- Inability to delete a node, given only a pointer to that node
- Inability to traverse the list in the backward direction

Header linked list

Introduction

- A header linked list is a linked list which always contains a special node called the *header node* at the beginning of the list.
- It is an extra node kept at the front of a list.
- Such a node does not represent an item in the list.
- The information portion might be unused or contains a flag.

Example



Simple list with a header Node

Header linked list cont..

- The external pointer to the list is to its header node
- The information portion of header node could be used to keep global information about the entire list such as:
- number of nodes (not including the header) in the list
- count in the header node must be adjusted after adding or deleting the item from the list
- pointer to the last node in the list

it simplifies the representation of a queue

pointer to the current node in the list

- eliminates the need of a external pointer during traversal

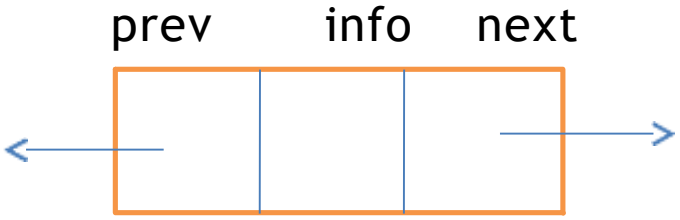
Doubly linked list

Introduction

It is a bidirectional list of structures(node) in which all nodes are linked together and contain three part.

- It can be either linear or circular and may or may not contains header node
- **INFO** part stores the information
- One **pointer** which points to the next element(successor)
- Other **pointer** points to the previous node (predecessor)

Example



A node of doubly linked list

Doubly linked list cont..

Representing doubly linked list nodes

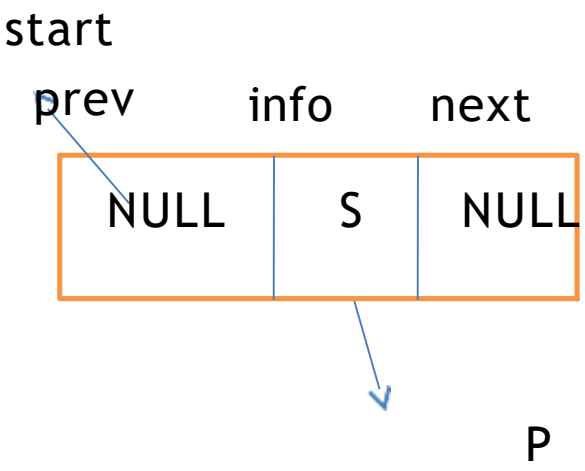
```
Struct node
{
Char info;
Struct node * prev;
Struct node * next;
};
typedef struct node NODE;
```

Doubly linked list cont..

Inserting and deleting in doubly linked list

Algorithm for inserting node at the beginning

allocate space for a new node using **malloc()**
copy the item into it,
p- ->info = S;
NODE * prev, * next;
If start = = NULL;
p - - > prev = p - - > next = NULL;
start = p;



Inserting and deleting in doubly linked list cont..

else

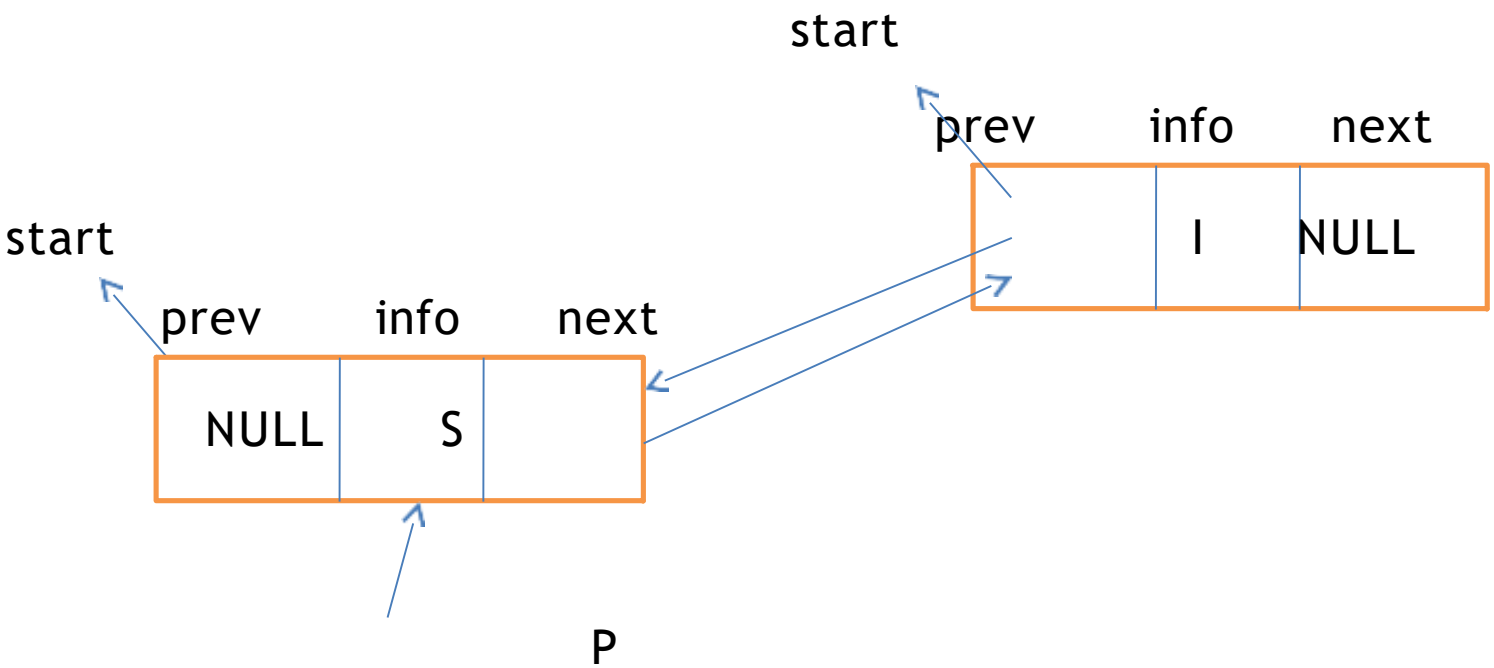
p - - > prev = NULL;

p - - > next = start;

start - - > prev = p;

start = p;

assume following list



Inserting and deleting in doubly linked list cont..

Deleting a node from the beginning of a doubly linked list

```
 If  start = = NULL;
return;
 else if
 start - - > next = = NULL;
 p = start;
start = NULL;
else
 p = start;
start = start - - > next;
start - - > prev = NULL;
free(p);
```

Advantage and disadvantage of circular linked list

Advantage

- Forward and backward accessing of nodes makes easy accessibility of nodes

- Insertion and deletions are easier and efficient than other list

Disadvantage

- It uses two pointers, so increases the memory requirement

T1ank /o1

H R Choudhary Asstt Professor Dept. Of CSE, Engineering College Ajmer